



University Asset Management System (ASM)

Technical Architecture & File Structure Analysis

SOMALI NATIONAL UNIVERSITY

A code-verified technical reference: every file, module, database relationship, workflow, role/permission rule, and security finding documented from the actual source code — prepared for developers and technical evaluators.

Companion document to: [University_Asset_Management_System_Documentation.pdf](#) (the end-user/admin functional manual)

Document Version 1.0

Table of Contents

Table of Contents	2
Part 1 — Complete Structure	4
1.1 Directory-by-Directory Role	4
1.2 How the Layers Communicate	5
Part 2 — File-by-File Analysis	6
2.1 Root	6
2.2 config/	6
2.3 database/	7
2.4 includes/	8
2.5 modules/auth/	14
2.6 modules/dashboard/	15
2.7 modules/assets/ — Asset Registry	16
2.8 modules/assigned/ — Allocations	19
2.9 modules/transfers/	20
2.10 modules/maintenance/	21
2.11 modules/audits/	23
2.12 modules/disposals/	24
2.13 modules/requisitions/	25
2.14 modules/categories/, modules/departments/, modules/locations/ — Master Data	27
2.15 modules/users/ — Account Management	29
2.16 modules/profile/	31
2.17 modules/reports/	31
2.18 modules/settings/ — Admin Configuration	33
2.19 static/ — Frontend Assets	38
2.20 vendor/fpdf/	40
2.21 docs/	41
Part 3 — Module Analysis	42
3.1 Assets Module	42
3.2 Authentication Module	43
3.3 Users Module	43
3.4 Departments, Categories, Locations Modules	43
3.5 Requisitions Module	43
3.6 Disposals Module	44
3.7 Maintenance Module	44

3.8 Transfers Module	44
3.9 Reports Module	44
3.10 Settings Module	45
3.11 Dashboard Module	45
3.12 Profile Module	45
Part 4 — Database Relationships	46
Part 5 — Role & Permission Analysis	48
Part 6 — Complete System Workflows	50
6.1 Authentication	50
6.2 Asset Management	50
6.3 Requisition	50
6.4 Disposal	50
6.5 Maintenance	50
6.6 Transfer	50
6.7 Reports — Generation and Export	51
6.8 Backup	51
Part 7 — Frontend	52
Part 8 — Authentication & Security	54
8.1 What Is Implemented Correctly (confirmed by reading the code)	54
8.2 Findings	54
8.3 Not Applicable / Not Found In This Codebase	57
Part 9 — Reports & PDF	58
Part 10 — Settings	59
Part 11 — Dependency Map	60
Part 12 — What Each File Contributes to the System	61
Part 13 — Missing or Weak Areas	64
Part 14 — Final Architecture Diagram	66
Final Recommendations	67
Complete Summary	67

Part 1 — Complete Structure

This section reconstructs the project structure directly from the files present on disk and explains the role of each top-level directory. Two notes on the structure as originally supplied for this analysis versus what actually exists in the repository:

- **Reconciled differences:** the module named `assigned/` (Allocations) is a top-level folder under `modules/`, not nested inside `modules/assets/`, and it has three files, not two — `add.php`, `list.php`, and `return.php` (the `return/repair` action). `includes/forbidden.php` actually lives at `includes/layout/forbidden.php`. There is no `static/img/` directory (only `static/images/logo.webp`). A root `index.php` entry point exists and was not listed. The FPDF font file names in `vendor/fpdf/font/` are `times.json` / `timesb.json` / `timesbi.json` / `timesi.json` etc. (the originally supplied tree had typos for three of these).
- **Pre-existing documentation:** `docs/University_Asset_Management_System_Documentation.pdf` and the Python script that generates it (`docs/build_pdf.py`) already exist in the repository as a functional, user/admin-facing manual. This document is a separate, deeper technical companion aimed at developers — it does not replace or duplicate that file.

1.1 Directory-by-Directory Role

Directory	Role in the System
<code>config/</code>	Two files loaded first by every request: database connection parameters (<code>database.php</code>) and global application configuration — base URL, session hardening, timezone, error-log routing (<code>config.php</code>).
<code>database/</code>	A single file, <code>schema.sql</code> : the complete MySQL schema (15 tables) plus demo seed data. This is the authoritative source of the database structure — there is no ORM or ODM; every table here is queried directly with hand-written SQL.
<code>docs/</code>	Project documentation: the existing user/admin manual PDF and its generator script, plus this technical analysis and its generator.
<code>includes/</code>	Shared backend logic loaded by (almost) every page: session/RBAC guards (<code>auth.php</code>), general helper functions (<code>functions.php</code>), the internationalization layer (<code>i18n.php</code> / <code>i18n_strings.php</code>), the single bootstrap include point (<code>bootstrap.php</code>), and the shared HTML shell (<code>includes/layout/*.php</code>).
<code>modules/</code>	One folder per feature area. Every PHP file in a module is a self-contained controller+view: it checks the caller's role, reads/writes the database with PDO, and renders its own HTML in the same file (no separate template engine).
<code>static/</code>	Presentation-only assets served directly by the web server: the CSS theme, two vanilla-JS files, and the bundled default logo. Nothing here executes server-side code.
<code>storage/</code>	Runtime, non-source content: the PHP error log (<code>php-error.log</code>) and a reserved (currently empty) <code>backups/</code> folder. Not meant to be served to browsers as content, though no explicit web-server rule in this codebase prevents that (see Part 8).
<code>uploads/</code>	User-supplied binary content written at runtime by the application: <code>uploads/logos</code> (admin-uploaded university logo) and <code>uploads/avatars</code> (user profile pictures). Both are populated by <code>move_uploaded_file()</code> calls in <code>modules/settings/index.php</code> and <code>modules/profile/index.php</code> respectively.
<code>vendor/</code>	One third-party library, vendored by hand (no Composer/package manager in this project): FPDF, used exclusively by <code>modules/reports/export_pdf.php</code> to generate downloadable PDF reports.

1.2 How the Layers Communicate

```
Browser (HTML forms / links, GET and POST)
|
v
modules/[area]/[file].php -- one file = one controller+view
| require_once includes/bootstrap.php (always first line)
v
includes/bootstrap.php
-> config/config.php (constants, session hardening, session_start())
-> config/database.php (creates $pdo : PDO, or dies with 500 on failure)
-> includes/auth.php (ROLE_* constants, isLoggedIn/requireLogin/requireRole/hasRole)
-> includes/functions.php (e, clean, csrf*, flash, logActivity, statusBadge,
recomputeAssetStatus, ...)
-> includes/i18n.php (t(), activeLanguage()); reads i18n_strings.php on first call)
-> setActiveLanguage(getSetting($pdo, 'language'))
-> maintenance-mode gate (blocks everyone except Admin if settings.maintenance_mode = '1')
|
v
the module file's own PHP logic
-> requireLogin() / requireRole([...]) (every protected page, before any output)
-> $pdo->prepare(...)->execute([...]) (all queries: bound parameters, never string-built SQL)
-> on POST: requireCsrf(), validate, write, logActivity(), flash(), redirect()
|
v
includes/layout/header.php (opens the HTML doc, loads static/css/style.css + Inter font,
includes sidebar.php, renders topbar incl. getPendingAlerts())
|
v
the module's own HTML output (uses e(), statusBadge(), formatMoney()/formatDate() from
functions.php)
|
v
includes/layout/footer.php (closes shell, loads static/js/main.js)
|
v
Browser renders page; static/js/main.js + validation.js add client-side interactivity
(all of it optional/UX-only – every rule is re-checked server-side on submit)
```

Every module file therefore has exactly the same top: `require_once bootstrap.php`, a role guard, then its own logic. There is no router, no MVC framework, and no template engine — the "View" is simply inline HTML in the same PHP file as the "Controller" logic, which is a deliberate, consistent choice throughout the codebase (confirmed by inspecting all ~50 module files).

Part 2 — File-by-File Analysis

Every PHP, SQL, JavaScript, CSS, and Python file that contributes to the running application is documented below, grouped by directory in the same order as Part 1. Each entry was produced by reading the file's actual contents. "Cannot be confirmed from the available code" is used explicitly wherever a question genuinely cannot be answered from what is in this repository.

2.1 Root

index.php

Purpose

The single public entry point for the whole application.

System Role

Router shim — redirects to the Dashboard if a session already exists, otherwise to the Login page.

Main Functions / Queries / Forms

- isLoggedIn()
- redirect()

Dependencies

- includes/bootstrap.php

Used By

- Loaded whenever a browser requests the site root ("/asm/" or "/asm/index.php")

User Roles

All (unauthenticated visitors included)

Workflow Context

First page hit for anyone visiting the site with no deep link; hands off immediately to Auth or Dashboard.

Importance: HIGH

Problems / Risks / Notes

None identified — it is a two-line, low-risk redirect.

2.2 config/

config/config.php

Purpose

Defines global constants and hardens/starts the PHP session before any other code runs.

System Role

Application bootstrap configuration — the very first file executed via bootstrap.php.

Main Functions / Queries / Forms

- Defines APP_NAME, APP_ROOT, APP_URL
- session.* ini_set() hardening (strict_mode, HttpOnly, SameSite=Lax, Secure-if-HTTPS)
- session_start()
- date_default_timezone_set()
- error_reporting/display_errors/log_errors to storage/php-error.log

Dependencies

- None (no includes)

Used By

- includes/bootstrap.php (first require)

User Roles

All

Workflow Context

Runs on literally every request before anything else — nothing works without it.

Importance: **CRITICAL**

Problems / Risks / Notes

- The session lifetime passed to `session_set_cookie_params()` is a hardcoded fallback (1440 minutes) read before the database/settings are available. The admin-configurable "Session Timeout (minutes)" setting saved in Settings → General is stored in the database but this file never re-applies it anywhere later in the request lifecycle — see Part 13.
- `APP_URL` is a hardcoded literal ("/asm") rather than derived from the request, so moving the deployment to a different sub-path requires editing this file.

config/database.php

Purpose

Opens the single shared PDO connection used by every module.

System Role

Data-access bootstrap — the only place a database connection is created.

Main Functions / Queries / Forms

- Defines `DB_HOST/DB_NAME/DB_USER/DB_PASS/DB_CHARSET`
- new `PDO(...)` with `ERRMODE_EXCEPTION, FETCH_ASSOC, EMULATE_PREPARES=false`

Dependencies

- None

Used By

- includes/bootstrap.php (second require); every module that touches `$pdo` depends on this indirectly

User Roles

All

Workflow Context

Runs on every request immediately after `config.php`.

Importance: **CRITICAL**

Problems / Risks / Notes

- Credentials are plain-text constants in a version-controlled file (root / empty password). Acceptable for the local/demo environment this project ships with, but must be replaced with environment-specific, non-committed credentials before any real deployment.
- `PDO::ATTR_EMULATE_PREPARES` is explicitly disabled, which is good for true server-side prepared statements — but it also means the same named placeholder cannot be bound twice in one query (`SQLSTATE[HY093]` if attempted). This exact mistake was found and fixed in `modules/auth/login.php` and `modules/assets/list.php` during earlier work on this codebase, and is still present and unfixed in `modules/users/list.php`'s search query — see Part 13.

2.3 database/

database/schema.sql

Purpose

The complete, authoritative MySQL schema (15 tables) plus demo seed data for all of them.

System Role

Data layer definition — every table every module reads or writes is defined here.

Main Functions / Queries / Forms

- CREATE DATABASE / 15x CREATE TABLE with explicit FOREIGN KEY ... ON UPDATE/ON DELETE rules
- INSERT seed data: 4 roles, 6 departments, 8 users (bcrypt-hashed demo password), 6 categories, 7 locations, 18 assets, and representative rows in every workflow table

Dependencies

- None (pure SQL, run once via `mysql -u root -p < database/schema.sql` per README.md)

Used By

- Every module file, indirectly, via the tables it creates

User Roles

DBA / Admin (setup-time only)

Workflow Context

Executed once to provision a fresh environment; not touched by the running application except indirectly via modules/settings/backup_download.php (re-generates an equivalent dump at runtime).

Importance: CRITICAL

Problems / Risks / Notes

- Comment block at the top documents a deliberate scope decision: the original brief specified 9-10 tables; this schema adds locations, asset_audits, settings, activity_logs, and login_logs to close gaps and support the admin Settings module — a documented, reasoned design choice, not scope creep.
- assets.status is designed to be derived (see includes/functions.php: recomputeAssetStatus()), but the schema itself does not enforce that with a trigger or generated column — a direct UPDATE from the Assets edit form (modules/assets/edit.php) can still set it out of sync with reality; the UI warns about this but does not prevent it.

2.4 includes/

includes/bootstrap.php

Purpose

The single include point every module entry file requires as its first line.

System Role

Composition root — wires config, database, auth, helpers, and i18n together in a fixed order, then applies the maintenance-mode gate.

Main Functions / Queries / Forms

- require_once config/config.php, config/database.php, auth.php, functions.php, i18n.php (in that order)
- setActiveLanguage(getSetting(\$pdo, "language", "en"))
- Maintenance-mode gate: if settings.maintenance_mode = "1" and the caller is logged in and not Admin, the request is stopped with an HTTP 503 page

Dependencies

- config/config.php
- config/database.php

- includes/auth.php
- includes/functions.php
- includes/i18n.php

Used By

- Every module/**/.php file in the project (the first line of each)

User Roles

All

Workflow Context

Runs on every authenticated and unauthenticated request alike; the maintenance-mode gate is the one piece of cross-cutting request-blocking logic in the whole system.

Importance: CRITICAL

Problems / Risks / Notes

None identified in the file itself; its correctness depends entirely on the files it requires.

includes/auth.php

Purpose

Defines the four role constants and every session/RBAC guard function used throughout the app.

System Role

Authorization core.

Main Functions / Queries / Forms

- ROLE_ADMIN/ROLE_OFFICER/ROLE_HEAD/ROLE_TOPMGMT constants
- isLoggedIn()
- currentUser()
- requireLogin() — redirects to login.php if not authenticated
- requireRole(array) — 403s via includes/layout/forbidden.php if the session role is not in the allowed list
- hasRole(array)
- isAdmin()

Dependencies

- \$_SESSION superglobal only

Used By

- Every module file (requireLogin()/requireRole()/hasRole() calls throughout)
- includes/functions.php (getPendingAlerts uses hasRole)
- includes/layout/sidebar.php (role-based menu rendering)

User Roles

All (defines the roles themselves)

Workflow Context

The gatekeeper for literally every restricted page and every conditional UI element in the system.

Importance: CRITICAL

Problems / Risks / Notes

- requireRole() correctly denies with an HTTP 403 page rather than silently redirecting, so a restricted URL visited directly is visibly blocked rather than confusingly bounced elsewhere — this is a good, deliberate security pattern, confirmed used consistently in every module file read.

includes/functions.php

Purpose

The general-purpose helper library shared by every module — sanitization, CSRF, formatting, activity logging, the notification-bell data source, and the shared report-query builder.

System Role

Cross-cutting utilities layer.

Main Functions / Queries / Forms

- e() / clean() — output escaping and input trimming
- redirect(), flash()
- csrfToken()/csrfField()/verifyCsrf()/requireCsrf()
- formatMoney()/formatDate()/formatDateTime()
- statusBadge()
- icon() — inline SVG icon library
- appLogoUrl()
- getPendingAlerts() — builds the topbar notification list
- getReportRows() — the single shared query builder used by both CSV and PDF report export
- logActivity()/logLogin()
- getAllSettings()/getSetting()/setSetting() — key/value settings cache
- recomputeAssetStatus() — derives assets.status from maintenance/disposal state
- validateRequired()

Dependencies

- \$pdo (PDO, passed in per-call)
- \$_SESSION
- includes/i18n.php (t(), activeLanguage())

Used By

- Every module file uses several of these helpers
- modules/reports/export_csv.php and export_pdf.php (getReportRows())
- includes/layout/header.php (getPendingAlerts(), appLogoUrl())

User Roles

All

Workflow Context

The most heavily depended-on file in the project after bootstrap.php itself.

Importance: CRITICAL

Problems / Risks / Notes

- recomputeAssetStatus() is the one piece of real "business logic" in the helper layer — it is called from modules/maintenance/add.php, modules/maintenance/update.php, and modules/disposals/approve.php, but not from modules/assets/edit.php, which lets an Admin/Officer hand-set a status that this function would otherwise immediately overwrite on the next maintenance/disposal event — a latent data-consistency trap flagged in Part 13.
- getPendingAlerts() looks back only 7 days for "your request was decided" notifications with no persisted read/unread state — see Part 13.

includes/i18n.php

Purpose

A minimal translation layer: resolves the active language once per request and exposes `t()/tWeekday()`.

System Role

Internationalization core.

Main Functions / Queries / Forms

- `setActiveLanguage(string)/activeLanguage()`
- `t(string $key)`: string — looks up `i18n_strings.php`, falls back to English then to the raw key
- `tWeekday(string $ymd)` — localized 3-letter weekday label

Dependencies

- `includes/i18n_strings.php` (lazily required on first `t()` call)

Used By

- `includes/bootstrap.php` (sets the active language)
- `includes/layout/*.php`
- `modules/auth/login.php`
- `modules/dashboard/index.php`
- `modules/settings/index.php` and `_subnav.php`
- `includes/functions.php` (`getPendingAlerts()` alert text)

User Roles

All

Workflow Context

Applied globally as soon as the site language setting is read in `bootstrap.php`.

Importance: **MEDIUM**

Problems / Risks / Notes

None in the mechanism itself — its only weakness is coverage (see `i18n_strings.php` below).

`includes/i18n_strings.php`

Purpose

The English/Somali translation dictionary consumed by `t()`.

System Role

Internationalization data.

Main Functions / Queries / Forms

- Returns a flat associative array: `key => ['en' => ..., 'so' => ...]`

Dependencies

- None

Used By

- `includes/i18n.php` only

User Roles

All (affects every user once Somali is selected in Settings → General)

Workflow Context

Read once per request (cached in a static variable inside `t()`) whenever any translated string is rendered.

Importance: **MEDIUM**

Problems / Risks / Notes

- The file's own header comment states coverage honestly: only the app shell (sidebar/topbar/footer), the login page, the dashboard, and Settings → General are translated. Every CRUD module (Assets, Allocations, Transfers, Maintenance, Audits, Disposals, Requisitions, Users, Departments, Categories, Locations, the other Settings tabs, Reports) remains English-only even with Somali active — a real, self-documented incomplete workflow (Part 13).

`includes/layout/header.php`

Purpose

Opens the shared HTML document, loads the theme CSS and Google Font, renders the topbar (notification bell + user menu), and starts the sidebar include.

System Role

Shared view shell — the "top half" of every authenticated page.

Main Functions / Queries / Forms

- Reads \$pageTitle/\$activeMenu set by the calling module
- getSetting() for university name and theme
- getPendingAlerts() for the bell dropdown
- includes sidebar.php
- renders flash() banners at the top of the <main> content area

Dependencies

- includes/functions.php
- includes/i18n.php
- includes/layout/sidebar.php
- static/css/style.css

Used By

- Every module file (via `include \_\_DIR\_\_ . '/../includes/layout/header.php`)

User Roles

All

Workflow Context

The first visual element rendered on every authenticated page.

Importance: CRITICAL

Problems / Risks / Notes

None identified — confirmed cache-busted (`?v=filetime(...)`) so CSS/JS edits are always picked up immediately rather than served stale from the browser cache.

`includes/layout/sidebar.php`

Purpose

Renders the left navigation menu, role-aware.

System Role

Shared view shell — primary navigation.

Main Functions / Queries / Forms

- navItem() helper
- Renders Administration section (Users/Departments/Categories/Locations/Settings) only when \$role === ROLE_ADMIN

Dependencies

- includes/functions.php (icon(), appLogoUrl())

- includes/i18n.php (t())

Used By

- includes/layout/header.php only

User Roles

All (content varies by role)

Workflow Context

Rendered as part of every authenticated page's shell.

Importance: HIGH

Problems / Risks / Notes

None identified — correctly relies on the same \$role check pattern as the server-side requireRole() guards, so the menu and the actual access rules never contradict each other.

includes/layout/footer.php

Purpose

Closes the shared HTML shell and loads the shared JavaScript file.

System Role

Shared view shell — the "bottom half" of every authenticated page.

Main Functions / Queries / Forms

- Renders the copyright line
- Loads static/js/main.js with a cache-busting query string

Dependencies

- includes/i18n.php (t())

Used By

- Every module file (via include at the end of its output)

User Roles

All

Workflow Context

The last thing rendered on every authenticated page.

Importance: HIGH

Problems / Risks / Notes

None identified.

includes/layout/forbidden.php

Purpose

A minimal standalone HTML page shown when requireRole() denies access.

System Role

Shared view shell — the 403 Access Denied page.

Main Functions / Queries / Forms

- Static HTML with a link back to the Dashboard

Dependencies

- static/css/style.css

Used By

- includes/auth.php: requireRole()
- modules/assets/view.php (direct 403 for a Department Head opening another department's asset)

User Roles

All (shown to whichever role was denied)

Workflow Context

The terminal page for any unauthorized direct-URL access attempt.

Importance: MEDIUM

Problems / Risks / Notes

Does not itself go through includes/layout/header.php, so it intentionally does not show the sidebar — reasonable, since the visitor was just denied access to that context.

2.5 modules/auth/

modules/auth/login.php

Purpose

The login form and its POST handler; the only unauthenticated page in the system besides the redirect shim.

System Role

Authentication entry point.

Main Functions / Queries / Forms

- GET: renders the login form
- POST: verifies CSRF, looks up `WHERE email = :login1 OR username = :login2`, password_verify(), checks status = "active", session_regenerate_id(true), populates \$_SESSION, logLogin() + logActivity(), redirects to \$_SESSION['redirect_after_login'] or the Dashboard

Dependencies

- includes/bootstrap.php
- includes/functions.php (logLogin, logActivity, csrfField/verifyCsrf)
- includes/i18n.php (all user-facing strings)
- static/js/validation.js

Used By

- index.php and includes/auth.php: requireLogin() (redirect target for anyone not logged in)

User Roles

Unauthenticated visitors only (already-logged-in users are redirected away)

Workflow Context

The first step of every session; sets every \$_SESSION key the rest of the app relies on.

Importance: CRITICAL

Problems / Risks / Notes

None identified in the current version — an earlier version of this query had a duplicate `:login` placeholder bound twice under EMULATE_PREPARES=false, which crashed with SQLSTATE[HY093]; it has since been fixed to use two distinct placeholders (:login1`/:login2`) bound to the same value. No login-attempt rate limiting or account lockout exists — see Part 8.

modules/auth/logout.php

Purpose

Ends the current session.

System Role

Authentication exit point.

Main Functions / Queries / Forms

- logActivity() if a session exists
- Clears \$_SESSION, expires the session cookie, session_destroy()
- Redirects to login.php

Dependencies

- includes/bootstrap.php
- includes/functions.php

Used By

- includes/layout/header.php (the Logout link in the user dropdown menu)

User Roles

Any logged-in user

Workflow Context

Terminates a session cleanly, including on the server side (not just clearing the cookie).

Importance: HIGH

Problems / Risks / Notes

None identified.

2.6 modules/dashboard/

`modules/dashboard/index.php`

Purpose

The role-scoped landing page shown immediately after login.

System Role

Summary/overview module.

Main Functions / Queries / Forms

- Six independent COUNT()/GROUP BY queries for the stat cards, status donuts, and 7-day activity chart (all department-scoped for a Department Head via \$isHead)
- renderDonut() — local helper building a CSS conic-gradient donut
- Recent Activity feed (12 most recent activity_logs rows, module-filtered for a Department Head)

Dependencies

- includes/bootstrap.php, functions.php, i18n.php
- Tables: assets, asset_assigned, asset_maintenance, asset_disposals, requisitions, users, activity_logs

Used By

- includes/auth.php: requireLogin() redirect target
- modules/auth/login.php (default post-login redirect)

User Roles

All (content scoped per role)

Workflow Context

The hub every user returns to; every number shown here is a live query, not a cached snapshot.

Importance: HIGH

Problems / Risks / Notes

None identified — all six queries were manually cross-checked against the same role-scoping pattern used consistently elsewhere in the codebase (\$isHead / department_id lock).

2.7 modules/assets/ — Asset Registry

modules/assets/list.php

Purpose

Searchable, filterable, paginated list of every asset.

System Role

Assets module — list/search view.

Main Functions / Queries / Forms

- Builds a dynamic WHERE clause from q/category_id/department_id/status/page
- Department Head is hard-locked to their own department_id regardless of query string
- Bound LIMIT/OFFSET pagination

Dependencies

- includes/bootstrap.php
- Tables: assets, categories, departments, locations

Used By

- includes/layout/sidebar.php (Assets nav link)
- Linked to from almost every other module's rows ("View asset")

User Roles

All (Admin/Officer see + edit; Department Head view-only, own department; Top Management view-only, all)

Workflow Context

The default browsing surface for the entire asset register.

Importance: CRITICAL

Problems / Risks / Notes

None currently — the search box previously reused the same named placeholder (':q`) twice in one `LIKE ... OR ... LIKE` clause, which crashed under EMULATE\_PREPARES=false; it has since been fixed to use `:q1`/:q2`. Default sort order was previously DESC (newest first); it is now ASC by asset_id (lowest ID first) as intentionally corrected.

modules/assets/add.php

Purpose

Registers a brand-new asset.

System Role

Assets module — create.

Main Functions / Queries / Forms

- Validates required fields + numeric cost + status enum
- INSERT into assets
- logActivity()
- Redirects straight to the new asset's view.php

Dependencies

- includes/bootstrap.php

- modules/assets/_form.php (shared field partial)
- Tables: assets, categories, departments, locations (dropdown sources)

Used By

- includes/layout/sidebar.php
- modules/assets/list.php ("Add Asset" button)

User Roles

Admin, Asset Officer

Workflow Context

Step one of the asset lifecycle: every other module (allocation, transfer, maintenance, audit, disposal) attaches its history to the row created here.

Importance: CRITICAL

Problems / Risks / Notes

None identified.

modules/assets/edit.php

Purpose

Edits an existing asset's details, including allowing a manual override of its status.

System Role

Assets module — update.

Main Functions / Queries / Forms

- Same validation as add.php
- UPDATE assets
- On error, re-merges posted values back into the form

Dependencies

- includes/bootstrap.php
- modules/assets/_form.php
- Tables: assets, categories, departments, locations

Used By

- modules/assets/list.php and view.php ("Edit" links)

User Roles

Admin, Asset Officer

Workflow Context

Corrections and reclassification within the asset lifecycle.

Importance: HIGH

Problems / Risks / Notes

The Status field can be hand-edited here even though the rest of the system treats it as derived (see `recomputeAssetStatus()` in `includes/functions.php`) — the form shows a warning label, but nothing server-side prevents an inconsistent manual value, which a subsequent maintenance/disposal event will then silently overwrite.

modules/assets/view.php

Purpose

The single detail page for one asset: its own fields plus five history tabs.

System Role

Assets module — detail/history view; the hub that ties every other module's per-asset records together.

Main Functions / Queries / Forms

- Five separate queries: `asset_assigned`, `asset_transfers`, `asset_maintenance`, `asset_audits`, `asset_disposals`, all ``WHERE asset_id = :id``
- Explicit 403 check: a Department Head whose department does not match `asset.department_id` is shown `includes/layout/forbidden.php` directly

Dependencies

- `includes/bootstrap.php`
- Tables: `assets` + all five history tables + `categories/departments/locations` for the header fields

Used By

- Linked to from `modules/assets/list.php` and from every other module's asset column (`assigned/transfers/maintenance/audits/disposals/requisitions` list rows)

User Roles

All (department-scoped for Department Head)

Workflow Context

The single source of truth for "everything that has ever happened to this asset."

Importance: **CRITICAL**

Problems / Risks / Notes

None identified — this is the one page in the system that most directly demonstrates the relational design (Part 4) by pulling from all five child tables in one view.

`modules/assets/_form.php`

Purpose

The shared HTML form fields reused by both `add.php` and `edit.php`.

System Role

Assets module — shared view partial (not a standalone page; has no `bootstrap.php` require of its own).

Main Functions / Queries / Forms

- Renders `Name/Category/Serial/Location/Department/Status/Purchase Date/Cost/Warranty/Description` fields

Dependencies

- Expects `$asset/$categories/$departments/$locations/$errors` to already be set by the including file

Used By

- `modules/assets/add.php`
- `modules/assets/edit.php`

User Roles

Admin, Asset Officer (the only roles that ever reach `add.php/edit.php`)

Workflow Context

Presentation partial for asset create/edit.

Importance: **MEDIUM**

Problems / Risks / Notes

None identified — a clean, intentional DRY pattern (one field partial, two controllers) that is not reused anywhere else in the codebase (every other module's `add/edit` forms are written out separately, e.g. `assigned/add.php` vs. `transfers/add.php`), so this is the one place the pattern was actually applied.

2.8 modules/assigned/ — Allocations

modules/assigned/add.php

Purpose

Issues (assigns) an asset to a department and, optionally, a named custodian.

System Role

Allocations module — create.

Main Functions / Queries / Forms

- Rejects a disposed asset
- Wraps the insert + the asset.department_id update in a DB transaction (beginTransaction/commit/rollBack)

Dependencies

- includes/bootstrap.php
- Tables: asset_assigned, assets, departments, users

Used By

- modules/assigned/list.php ("+ Assign Asset")

User Roles

Admin, Asset Officer

Workflow Context

The step that turns "the university owns this" into "this department currently holds this."

Importance: HIGH

Problems / Risks / Notes

None identified — the explicit transaction here (and in transfers/add.php) is a deliberately more careful pattern than most other write operations in the codebase, appropriate since two related tables (asset_assigned + assets) must stay consistent.

modules/assigned/list.php

Purpose

Filterable list of every allocation, current and historical, with "Mark Returned"/"Send to Repair" actions on active rows.

System Role

Allocations module — list/action view.

Main Functions / Queries / Forms

- Filter by department (locked for Department Head)/custodian/status
- Two POST forms per active row targeting return.php with a different `outcome` hidden field

Dependencies

- includes/bootstrap.php
- Tables: asset_assigned, assets, departments, users

Used By

- includes/layout/sidebar.php

User Roles

All (Admin/Officer act; Department Head + Top Management view-only)

Workflow Context

The operational view of "who has what right now."

Importance: HIGH

Problems / Risks / Notes

None identified.

modules/assigned/return.php

Purpose

POST-only endpoint that closes an active allocation as either Returned or Repair.

System Role

Allocations module — state transition.

Main Functions / Queries / Forms

- `UPDATE asset\_assigned SET return\_date = today, status = :outcome WHERE assign\_id = :id AND status = "active"`

Dependencies

- includes/bootstrap.php

Used By

- modules/assigned/list.php (the two per-row forms)

User Roles

Admin, Asset Officer

Workflow Context

Closes the loop on an allocation; does not itself touch assets.status (only Maintenance/Disposals do that).

Importance: MEDIUM

Problems / Risks / Notes

Redirects to list.php on any non-POST request rather than 404ing, which is a minor but consistent defensive pattern used by every similar action-only endpoint in the codebase (return.php, deactivate.php).

2.9 modules/transfers/

modules/transfers/add.php

Purpose

Moves an asset from its current department to a new one.

System Role

Transfers module — create.

Main Functions / Queries / Forms

- Rejects a disposed asset or a "transfer" to the asset's current department
- Transaction wrapping the insert + assets.department_id update

Dependencies

- includes/bootstrap.php
- Tables: asset_transfers, assets, departments

Used By

- modules/transfers/list.php

User Roles

Admin, Asset Officer

Workflow Context

Records inter-department movement separately from day-to-day custodian allocation.

Importance: **HIGH**

Problems / Risks / Notes

None identified.

modules/transfers/list.php

Purpose

Full transfer history, department-scoped (source or destination) for a Department Head.

System Role

Transfers module — list view.

Main Functions / Queries / Forms

- `WHERE from\_department\_id = :d OR to\_department\_id = :d` for a Department Head

Dependencies

- includes/bootstrap.php
- Tables: asset_transfers, assets, departments, users

Used By

- includes/layout/sidebar.php

User Roles

All (Admin/Officer act; Department Head + Top Management view-only)

Workflow Context

Read-only audit trail of every inter-department move.

Importance: **MEDIUM**

Problems / Risks / Notes

None identified.

2.10 modules/maintenance/

modules/maintenance/add.php

Purpose

Reports a maintenance issue against an asset.

System Role

Maintenance module — create (the one create-form open to Department Head as well as Admin/Officer).

Main Functions / Queries / Forms

- Department Head's asset dropdown is pre-filtered to their own department; the server independently re-checks the department match on submit
- INSERT with status "pending"
- recomputeAssetStatus() called immediately after

Dependencies

- includes/bootstrap.php
- includes/functions.php (recomputeAssetStatus)
- Tables: asset_maintenance, assets

Used By

- modules/maintenance/list.php

User Roles

Admin, Asset Officer, Department Head

Workflow Context

The entry point that (indirectly) flips an asset to "Under Repair."

Importance: HIGH

Problems / Risks / Notes

None identified.

modules/maintenance/list.php

Purpose

Filterable list of maintenance tickets.

System Role

Maintenance module — list view.

Main Functions / Queries / Forms

- Filter by status
- Department-scoped for Department Head

Dependencies

- includes/bootstrap.php
- Tables: asset_maintenance, assets, users

Used By

- includes/layout/sidebar.php

User Roles

All (Admin/Officer manage; Department Head reports + views; Top Management view-only)

Workflow Context

Operational queue of open and historical repair tickets.

Importance: HIGH

Problems / Risks / Notes

None identified.

modules/maintenance/update.php

Purpose

Progresses a ticket through Pending → In Progress → Completed and records cost/technician.

System Role

Maintenance module — update/close.

Main Functions / Queries / Forms

- Requires a Completed Date when status is set to "completed"
- Validates Cost is numeric
- recomputeAssetStatus() called after every save, which is what returns an asset to "Active" once all its open tickets are closed

Dependencies

- includes/bootstrap.php
- includes/functions.php
- Tables: asset_maintenance, assets

Used By

- modules/maintenance/list.php ("Update" link)

User Roles

Admin, Asset Officer

Workflow Context

The step that actually closes the maintenance loop and restores the asset's status.

Importance: HIGH

Problems / Risks / Notes

None identified.

2.11 modules/audits/

modules/audits/add.php

Purpose

Records a physical stock-check result for an asset.

System Role

Audits module — create.

Main Functions / Queries / Forms

- INSERT into asset_audits with result IN (found, missing, damaged)

Dependencies

- includes/bootstrap.php
- Tables: asset_audits, assets

Used By

- modules/audits/list.php

User Roles

Admin, Asset Officer

Workflow Context

A deliberately isolated workflow — recording Missing/Damaged does not itself change assets.status.

Importance: MEDIUM

Problems / Risks / Notes

None identified — the asset dropdown here intentionally includes disposed assets (unlike every other create form), since an audit can legitimately confirm a disposed asset is indeed gone; this is a correct, deliberate difference, not an oversight.

modules/audits/list.php

Purpose

Read-only, department-scoped list of every audit record.

System Role

Audits module — list view.

Main Functions / Queries / Forms

- Department-scoped WHERE for Department Head

Dependencies

- includes/bootstrap.php

- Tables: asset_audits, assets, users

Used By

- includes/layout/sidebar.php

User Roles

All (Admin/Officer create; everyone else view-only)

Workflow Context

Permanent physical-verification history; no edit/delete exists for audit rows anywhere in the codebase.

Importance: MEDIUM

Problems / Risks / Notes

None identified.

2.12 modules/disposals/

modules/disposals/add.php

Purpose

Requests the disposal of an asset (does not dispose it).

System Role

Disposals module — create request.

Main Functions / Queries / Forms

- Rejects an already-disposed asset or one with an existing pending disposal request

Dependencies

- includes/bootstrap.php
- Tables: asset_disposals, assets

Used By

- modules/disposals/list.php

User Roles

Admin, Asset Officer

Workflow Context

Step 1 of the two-party disposal workflow — deliberately cannot approve its own request.

Importance: HIGH

Problems / Risks / Notes

None identified.

modules/disposals/approve.php

Purpose

Approves or rejects a pending disposal request.

System Role

Disposals module — the sole point where an asset actually becomes "Disposed."

Main Functions / Queries / Forms

- Refuses to open an already-reviewed request
- Transaction wrapping the status update + recomputeAssetStatus() (only called on approval)

Dependencies

- includes/bootstrap.php
- includes/functions.php
- Tables: asset_disposals, assets

Used By

- modules/disposals/list.php ("Review" link, Admin/Top Management only)

User Roles

Admin, Top Management

Workflow Context

Step 2 of the disposal workflow — the deliberate separation-of-duties control described in the README.

Importance: HIGH

Problems / Risks / Notes

None identified — role gating (Admin/Top Management only, explicitly excluding the Asset Officer who requested it) is a correctly-enforced separation of duties, confirmed in both the role guard and the permission matrix in Part 5.

modules/disposals/list.php

Purpose

Filterable list of disposal requests and their outcomes.

System Role

Disposals module — list view.

Main Functions / Queries / Forms

- Filter by status
- Department-scoped for Department Head

Dependencies

- includes/bootstrap.php
- Tables: asset_disposals, assets, users

Used By

- includes/layout/sidebar.php

User Roles

All (Admin requests+approves; Officer requests only; Department Head view-only; Top Management approves)

Workflow Context

The full history of every write-off decision made in the system.

Importance: HIGH

Problems / Risks / Notes

None identified.

2.13 modules/requisitions/

modules/requisitions/add.php

Purpose

Submits a department's request for new/replacement assets.

System Role

Requisitions module — create.

Main Functions / Queries / Forms

- Department Head's department field is locked to their own session department_id

Dependencies

- includes/bootstrap.php
- Tables: requisitions, departments, categories

Used By

- modules/requisitions/list.php

User Roles

Admin, Department Head

Workflow Context

The starting point of the requisition workflow; does not itself touch the assets table.

Importance: HIGH

Problems / Risks / Notes

None identified.

modules/requisitions/list.php

Purpose

Filterable list of requisitions and their status.

System Role

Requisitions module — list view.

Main Functions / Queries / Forms

- Filter by status
- Department-scoped for Department Head

Dependencies

- includes/bootstrap.php
- Tables: requisitions, departments, categories, users

Used By

- includes/layout/sidebar.php

User Roles

All (Admin/Officer review; Department Head submits+views own; Top Management view-only)

Workflow Context

The queue Asset Officers/Admin work from when deciding what to approve.

Importance: HIGH

Problems / Risks / Notes

The search query joining `u.name LIKE :q OR u.email LIKE :q OR u.username LIKE :q` for a very similar search box exists on modules/users/list.php, not here — see that entry for the still-open bug.

modules/requisitions/review.php

Purpose

Moves a requisition through its status workflow: Pending → Approved/Rejected, Approved → Issued.

System Role

Requisitions module — review/decision.

Main Functions / Queries / Forms

- \$allowedTransitions map strictly enforces which decision is valid for the current status; anything else is rejected with "Invalid action for the current status of this requisition."

Dependencies

- includes/bootstrap.php
- Tables: requisitions

Used By

- modules/requisitions/list.php ("Review" / "Mark Issued" links)

User Roles

Admin, Asset Officer

Workflow Context

The decision point of the requisition workflow.

Importance: HIGH

Problems / Risks / Notes

Unlike Disposals, an Asset Officer can approve/reject/issue a requisition without any Department Head or Top Management sign-off — a different, less strict governance model than Disposals. This appears to be an intentional design choice (requisitions are lower-stakes than write-offs) but is worth confirming against the intended business process — see Part 13.

2.14 modules/categories/, modules/departments/, modules/locations/ — Master Data

`modules/categories/list.php`

Purpose

Full CRUD for asset categories in a single file, using modal dialogs for create/edit.

System Role

Master data module.

Main Functions / Queries / Forms

- create/update/delete actions dispatched by a hidden `action` POST field
- Delete wrapped in try/catch so an FK-constrained row (assets still referencing it) fails gracefully with a friendly flash message instead of a raw SQL error

Dependencies

- includes/bootstrap.php
- Tables: categories, assets (for the per-row asset_count subquery)

Used By

- includes/layout/sidebar.php (Administration section, Admin only)

User Roles

Admin only

Workflow Context

Feeds the Category dropdown used by Assets and Requisitions.

Importance: MEDIUM

Problems / Risks / Notes

None identified.

modules/departments/list.php

Purpose

Full CRUD for departments, including assigning a Head of Department.

System Role

Master data module.

Main Functions / Queries / Forms

- Same create/update/delete pattern as categories/list.php
- Head-of-Department dropdown sourced from `users WHERE role\_id IN (SELECT ... WHERE role\_name = 'Department Head')`

Dependencies

- includes/bootstrap.php
- Tables: departments, users, assets (asset_count subquery)

Used By

- includes/layout/sidebar.php (Admin only)

User Roles

Admin only

Workflow Context

Defines the department_id every other module scopes a Department Head's view by.

Importance: CRITICAL

Problems / Risks / Notes

None identified — though because department_id drives every single "\$isHead" scoping check across the entire application, any data error introduced here has the widest possible blast radius of any master data module.

modules/locations/list.php

Purpose

Full CRUD for physical building/room locations.

System Role

Master data module.

Main Functions / Queries / Forms

- Same create/update/delete modal pattern as the other two master-data modules

Dependencies

- includes/bootstrap.php
- Tables: locations, assets (asset_count subquery)

Used By

- includes/layout/sidebar.php (Admin only)

User Roles

Admin only

Workflow Context

Feeds the Location dropdown used by the Assets module.

Importance: MEDIUM

Problems / Risks / Notes

None identified.

2.15 modules/users/ — Account Management

modules/users/add.php

Purpose

Creates a new user account (the only way new accounts are created — there is no public sign-up).

System Role

User administration module.

Main Functions / Queries / Forms

- Validates name/email/username/role
- Uniqueness checks on email and username (separate SELECT 1 queries)
- password_hash() for the temporary password

Dependencies

- includes/bootstrap.php
- Tables: users, roles, departments

Used By

- modules/users/list.php

User Roles

Admin only

Workflow Context

Provisions every account other than the ones in the seed data.

Importance: CRITICAL

Problems / Risks / Notes

None identified.

modules/users/edit.php

Purpose

Edits a user's profile fields, role, department, and optionally resets their password.

System Role

User administration module.

Main Functions / Queries / Forms

- Same validation/uniqueness pattern as add.php
- Password field is optional — blank leaves the existing hash untouched

Dependencies

- includes/bootstrap.php
- Tables: users, roles, departments

Used By

- modules/users/list.php ("Edit")

User Roles

Admin only

Workflow Context

The mechanism by which an Admin can promote/demote a role or move someone between departments.

Importance: CRITICAL

Problems / Risks / Notes

None identified.

modules/users/list.php

Purpose

Searchable, filterable list of every user account with Deactivate/Reactivate actions.

System Role

User administration module.

Main Functions / Queries / Forms

- Search across name/email/username
- Filter by role

Dependencies

- includes/bootstrap.php
- Tables: users, roles, departments

Used By

- includes/layout/sidebar.php (Admin only)

User Roles

Admin only

Workflow Context

The operational hub for account management.

Importance: CRITICAL

Problems / Risks / Notes

The search clause `(u.name LIKE :q OR u.email LIKE :q OR u.username LIKE :q)` reuses the same named placeholder `:q` three times in one query. Under this project's PDO configuration (EMULATE_PREPARES = false, in config/database.php), a repeated named placeholder is invalid and throws `SQLSTATE[HY093]: Invalid parameter number` — meaning searching the Users list currently crashes with an HTTP 500 for any non-empty search term. This is the same bug class already found and fixed in modules/auth/login.php and modules/assets/list.php, but this occurrence is still present. See Part 13 for the concrete fix.

modules/users/deactivate.php

Purpose

POST-only endpoint toggling a user between active/inactive.

System Role

User administration module — state transition.

Main Functions / Queries / Forms

- Refuses to let an Admin deactivate their own account

Dependencies

- includes/bootstrap.php
- Tables: users

Used By

- modules/users/list.php (per-row Deactivate/Reactivate form)

User Roles

Admin only

Workflow Context

An inactive account is refused at login (see modules/auth/login.php) without deleting any of its history.

Importance: HIGH

Problems / Risks / Notes

None identified.

2.16 modules/profile/

`modules/profile/index.php`

Purpose

Self-service page for any logged-in user: edit their own name/email/username/profile picture, and change their own password.

System Role

Self-service account module.

Main Functions / Queries / Forms

- Two independent forms distinguished by a `form\_type` hidden field (profile vs. password)
- Profile-picture upload: extension whitelist (jpg/jpeg/png/gif) + 2MB size cap + random filename via `bin2hex(random_bytes(6))`; old file deleted on replace/remove
- Password change requires the correct current password

Dependencies

- includes/bootstrap.php
- Tables: users

Used By

- includes/layout/header.php (the "My Profile" link in every page's user menu)

User Roles

All (each user can only edit their own account)

Workflow Context

The only place a non-Admin can change any of their own account details.

Importance: HIGH

Problems / Risks / Notes

File-upload validation checks extension and size but not actual image content (e.g. via `getimagesize()` or `finfo` mime-type detection); see Part 8 for the associated risk and recommended fix.

2.17 modules/reports/

`modules/reports/index.php`

Purpose

The on-screen Reports UI: five report types, each with a clickable Asset Count that opens the exact underlying asset records in a modal.

System Role

Reporting module — on-screen view.

Main Functions / Queries / Forms

- Five report branches (by_department/by_category/by_status/maintenance_cost/disposals), each pairing an aggregate GROUP BY query with a matching per-row detail query scoped identically
- renderAssetCountCell()/renderAssetCountTrigger()/renderAssetDetailModal() — local helpers building the clickable-count modal UI
- The "Assets by Department" report additionally makes the whole table row clickable (not just the count), with its modals rendered after the table rather than nested inside the row it belongs to, specifically to avoid a close-button/re-open interaction bug

Dependencies

- includes/bootstrap.php
- Tables: departments, categories, assets, locations, asset_maintenance, asset_disposals

Used By

- includes/layout/sidebar.php

User Roles

All (Admin/Officer full; Department Head department-scoped; Top Management full, view-only)

Workflow Context

The system's analytics surface; every number here is a live query, and the detail modal is proven (by construction) to reflect the exact same rows the count was computed from, not a separate "first N" query.

Importance: HIGH

Problems / Risks / Notes

None identified in the current version — an earlier iteration nested each modal inside its own trigger row, which would have made the modal's own close button re-trigger the row's click handler and immediately reopen it; this was identified and restructured before shipping.

modules/reports/export_csv.php

Purpose

Streams any of the five reports as a CSV download.

System Role

Reporting module — export.

Main Functions / Queries / Forms

- Delegates entirely to includes/functions.php: getReportRows()
- fputcsv() to php://output

Dependencies

- includes/bootstrap.php
- includes/functions.php

Used By

- modules/reports/index.php ("Export CSV" button, carries the current filters via the query string)

User Roles

Same as modules/reports/index.php

Workflow Context

Offline/spreadsheet consumption of report data.

Importance: MEDIUM

Problems / Risks / Notes

None identified.

modules/reports/export_pdf.php

Purpose

Streams any of the five reports as a formatted PDF download.

System Role

Reporting module — export.

Main Functions / Queries / Forms

- Delegates data to includes/functions.php: getReportRows()
- ReportPDF extends FPDF, with a branded Header()/Footer() and a fitText() helper that truncates any cell text with an ellipsis so it can never overflow a fixed-width column
- Auto-selects Landscape orientation once a report has 4+ columns

Dependencies

- includes/bootstrap.php
- includes/functions.php
- vendor/fpdf/fpdf.php

Used By

- modules/reports/index.php ("Export PDF" button)

User Roles

Same as modules/reports/index.php

Workflow Context

Printable/archivable report output, without any dependency beyond the vendored FPDF library.

Importance: MEDIUM

Problems / Risks / Notes

None identified.

2.18 modules/settings/ — Admin Configuration

modules/settings/index.php

Purpose

General system settings: university/system name, academic year, language, timezone, date format, session timeout, records-per-page, theme, logo upload, and the maintenance-mode switch.

System Role

Settings module — General tab.

Main Functions / Queries / Forms

- Validates + saves each field via setSetting()
- Logo upload: same extension/size validation pattern as the profile-picture uploader

Dependencies

- includes/bootstrap.php
- includes/i18n.php
- modules/settings/_subnav.php
- Table: settings

Used By

- includes/layout/sidebar.php (Admin only)

User Roles

Admin only

Workflow Context

The single source for every `getSetting()` call made anywhere else in the app (university name, theme, language, records-per-page, maintenance mode, logo).

Importance: **CRITICAL**

Problems / Risks / Notes

Two settings saved here are not actually enforced anywhere else in the codebase: ``session_timeout_minutes`` (the real session cookie lifetime is fixed in `config/config.php`, read before this setting could apply) and, more broadly, changes to it have no observable effect. See Part 13.

modules/settings/smtp.php

Purpose

Collects SMTP host/port/username/password/encryption for "outgoing notification emails."

System Role

Settings module — Email tab.

Main Functions / Queries / Forms

- Validates port is numeric
- Saves each field via `setSetting()`; password only overwritten if a new one is entered

Dependencies

- `includes/bootstrap.php`
- `modules/settings/_subnav.php`
- Table: settings

Used By

- `includes/layout/sidebar.php` (Admin only)

User Roles

Admin only

Workflow Context

Intended to back an email-notification feature.

Importance: **MEDIUM**

Problems / Risks / Notes

- A codebase-wide search finds no `PHPMailer/SwiftMailer/mail()` call or any other consumer of these settings anywhere in the project — these credentials are saved but never used to send anything. The page's own help text ("Used for outgoing notification emails, e.g. maintenance updates, requisition decisions") describes a capability that does not exist yet. See Part 13.

modules/settings/backup.php

Purpose

Landing page for database backup/restore: shows DB size/table count, and links to the download and restore actions.

System Role

Settings module — Backup & Restore tab.

Main Functions / Queries / Forms

- ``SELECT SUM(data_length+index_length) ... information_schema.TABLES`` for the size stat

Dependencies

- includes/bootstrap.php
- modules/settings/_subnav.php

Used By

- modules/settings/backup_download.php
- modules/settings/backup_restore.php

User Roles

Admin only

Workflow Context

Entry point for the one genuinely destructive workflow in the whole system.

Importance: HIGH

Problems / Risks / Notes

None in this file itself — see backup_restore.php for the associated critical-severity risk.

modules/settings/backup_download.php

Purpose

Streams a complete .sql dump of the live database (schema + all data), generated in pure PHP.

System Role

Settings module — Backup & Restore tab, download action.

Main Functions / Queries / Forms

- SHOW TABLES, then per table: SHOW CREATE TABLE + SELECT * with hand-built INSERT statements (\$pdo->quote() used for every value, so this is not raw string concatenation of untrusted input)

Dependencies

- includes/bootstrap.php

Used By

- modules/settings/backup.php ("Download Backup" button)

User Roles

Admin only

Workflow Context

The system's only backup mechanism (no scheduled/automatic backup job exists anywhere in the codebase).

Importance: HIGH

Problems / Risks / Notes

The generated dump includes every user's bcrypt password hash and the raw smtp_password setting value (if one has been set) in plain INSERT statements — reasonable for a database backup, but it means the downloaded file itself becomes sensitive and must be stored as carefully as the live database.

modules/settings/backup_restore.php

Purpose

Accepts an uploaded .sql file and executes its contents directly against the live database.

System Role

Settings module — Backup & Restore tab, restore action.

Main Functions / Queries / Forms

- Validates extension is .sql and size ≤ 50MB

- `pdo->exec(\$sql)`, where \$sql is the entire raw uploaded file content

Dependencies

- includes/bootstrap.php

Used By

- modules/settings/backup.php (the Restore form)

User Roles

Admin only

Workflow Context

A full-database overwrite mechanism.

Importance: CRITICAL

Problems / Risks / Notes

- This executes arbitrary, attacker-or-mistake-controlled SQL directly against the production database with no dry-run, no automatic safety backup taken first, and no statement-level filtering — the only gate is the file extension and an Admin-role check. Any admin session (whether the legitimate admin or an attacker who has compromised that session, e.g. via a stolen cookie or CSRF if the token were ever missing) can drop/recreate/repopulate every table in one request. See Part 8 for the full write-up.

modules/settings/logs.php

Purpose

Paginated, module-filterable view of the system-wide activity_logs table.

System Role

Settings module — Activity Logs tab.

Main Functions / Queries / Forms

- Bound LIMIT/OFFSET pagination (25/page)

Dependencies

- includes/bootstrap.php
- modules/settings/_subnav.php
- Table: activity_logs

Used By

- includes/layout/sidebar.php (Admin only)

User Roles

Admin only

Workflow Context

The audit trail for every logActivity() call made anywhere in the system (every module calls it after a create/update/delete/state-change).

Importance: HIGH

Problems / Risks / Notes

None identified.

modules/settings/login_logs.php

Purpose

Paginated, status-filterable view of every login attempt, successful or failed.

System Role

Settings module — Login Logs tab.

Main Functions / Queries / Forms

- Bound LIMIT/OFFSET pagination (25/page)

Dependencies

- includes/bootstrap.php
- modules/settings/_subnav.php
- Table: login_logs

Used By

- includes/layout/sidebar.php (Admin only)

User Roles

Admin only

Workflow Context

The forensic trail for investigating unauthorized-access attempts — every logLogin() call from modules/auth/login.php lands here.

Importance: HIGH

Problems / Risks / Notes

None identified — though see Part 8: nothing currently acts on repeated failures recorded here (no automatic logout), so it is presently observational only.

modules/settings/system_info.php

Purpose

Read-only environment snapshot: PHP/MySQL version, server software, OS, disk space.

System Role

Settings module — System Info tab.

Main Functions / Queries / Forms

- PHP_VERSION/PHP_OS constants
- `SELECT VERSION()`
- disk_free_space()/disk_total_space()
- Local humanSize() helper

Dependencies

- includes/bootstrap.php
- modules/settings/_subnav.php

Used By

- includes/layout/sidebar.php (Admin only)

User Roles

Admin only

Workflow Context

A diagnostic page for technical support/evaluation, with no write operations at all.

Importance: LOW

Problems / Risks / Notes

None identified.

modules/settings/_subnav.php

Purpose

The shared sub-navigation tab strip rendered at the top of every Settings page.

System Role

Settings module — shared view partial.

Main Functions / Queries / Forms

- \$settingsTabs array driving the tab links

Dependencies

- includes/i18n.php (t())

Used By

- Every modules/settings/*.php file except backup_download.php/backup_restore.php (which are pure redirect/stream endpoints with no HTML output)

User Roles

Admin only

Workflow Context

Keeps the six Settings pages visually and structurally consistent.

Importance: LOW

Problems / Risks / Notes

None identified.

2.19 static/ — Frontend Assets

static/css/style.css

Purpose

The entire visual theme: layout, components, and both the light and dark color schemes.

System Role

Frontend presentation layer.

Main Functions / Queries / Forms

- CSS custom properties (:root) for the light palette, re-pointed under `body.theme-dark` for the dark palette — components reference the variables, not hardcoded colors, so both themes are produced by one shared rule set
- Sidebar/topbar/cards/tables/forms/badges/modals/tabs/pagination/auth-page/dashboard-insight/report-modal component styles
- Two responsive breakpoints (992px, 600px)

Dependencies

- Google Fonts "Inter" (loaded via a <link> tag in header.php, not bundled locally)

Used By

- includes/layout/header.php and modules/auth/login.php (both load it with a filemtime() cache-busting query string)

User Roles

All

Workflow Context

Every page in the system shares this one stylesheet — there is no per-module CSS.

Importance: HIGH

Problems / Risks / Notes

- The dark theme is implemented by re-defining the shared CSS custom properties under ``body.theme-dark`` (rather than one override rule per component), so any new component automatically supports both themes without further CSS work — a deliberate architectural choice made after an earlier version of the dark theme used per-component overrides with stale, pre-redesign colors.

`static/js/main.js`

Purpose

All shared client-side interactivity: sidebar toggle, dropdown menus, generic modal open/close, destructive-action confirmations, client-side table search/sort, and tab switching.

System Role

Frontend behavior layer.

Main Functions / Queries / Forms

- Entirely attribute-driven (``data-dropdown-toggle``, ``data-modal-target``, ``data-modal-close``, ``data-confirm``, ``data-table-search``, ``data-sort``, ``.tab-link``) — no per-page custom JS exists anywhere in the project
- One generic click-outside-closes-dropdown handler; one generic modal system reused by every module that has modals (categories/departments/locations CRUD, reports asset-count detail)

Dependencies

- None (vanilla JS, no build step, no framework)

Used By

- `includes/layout/footer.php` (loaded on every authenticated page with a cache-busting query string)

User Roles

All

Workflow Context

Every interactive widget in the authenticated app depends on this one file.

Importance: **HIGH**

Problems / Risks / Notes

None identified — the attribute-driven, single-shared-script approach is consistently used everywhere; no module was found to define its own bespoke inline `<script>` interactivity outside of this file.

`static/js/validation.js`

Purpose

Client-side form validation: required fields, email format, numeric min/max, minlength.

System Role

Frontend behavior layer — UX convenience only.

Main Functions / Queries / Forms

- `validateField()/showError()/clearError()`
- Wires every `<form>` on the page automatically on submit and on blur

Dependencies

- None

Used By

- `modules/auth/login.php` (the only page observed to load it directly instead of relying on `footer.php`)

User Roles

All

Workflow Context

The file's own header comment states its role precisely: "This is a UX convenience layer only — every form is re-validated server-side in PHP before touching the database" — confirmed true by inspecting every module's POST handler, which all re-run `validateRequired()` and their own checks regardless of this script.

Importance: MEDIUM

Problems / Risks / Notes

None identified.

`static/images/logo.webp`

Purpose

The bundled default university crest/logo.

System Role

Frontend asset.

Main Functions / Queries / Forms

- Served as-is; not processed by PHP

Dependencies

- None

Used By

- `includes/functions.php: appLogoUrl()` falls back to this file whenever no admin-uploaded logo (`settings.logo_path`) is set

User Roles

All

Workflow Context

Brand identity shown in the sidebar, browser tab icon, and login page whenever no custom logo has been uploaded.

Importance: LOW

Problems / Risks / Notes

None identified.

2.20 vendor/fpdf/

`vendor/fpdf/fpdf.php (+ font/*.json, LICENSE.txt)`

Purpose

A hand-vendored copy of the FPDF library (no Composer/package manager used anywhere in this project).

System Role

Third-party dependency — the only one in the codebase.

Main Functions / Queries / Forms

- Standard FPDF class API (`AddPage/Cell/SetFont/SetFillColor/Output/etc.`); the 14 core-font metric files in `font/` back Helvetica/Times/Courier/Symbol/ZapfDingbats in regular/bold/italic/bold-italic

Dependencies

- None (self-contained; no Composer autoloader involved)

Used By

- `modules/reports/export_pdf.php` (the only consumer in the project)

User Roles

Same as `modules/reports/export_pdf.php`

Workflow Context

The PDF-rendering engine behind Reports → Export PDF.

Importance: MEDIUM

Problems / Risks / Notes

None identified — vendoring by hand (rather than via Composer) is consistent with the rest of the project's "no framework, no package manager" approach, and keeps the dependency footprint to exactly one library used for exactly one feature.

2.21 docs/

`docs/build_pdf.py`

Purpose

Generates the existing end-user/admin functional manual, `University_Asset_Management_System_Documentation.pdf`, using `reportlab`.

System Role

Documentation tooling (not part of the running web application).

Main Functions / Queries / Forms

- Data-driven `module_section()/data_table()/module_header()` helpers producing a full user manual with a cover page, table of contents, and one detailed section per module

Dependencies

- Python: `reportlab`
- A hardcoded absolute path to a logo image

Used By

- Run manually/offline by a developer; not invoked by any PHP code

User Roles

Developer/documentation maintainer only

Workflow Context

Produces the companion functional manual referenced throughout this document.

Importance: LOW

Problems / Risks / Notes

- LOGO is a hardcoded absolute Windows path, ``C:\xampp\htdocs\asm\storage\logo_converted.png``, which does not currently exist in the repository's `storage/` directory (only ``.gitkeep`` and ``.php-error.log`` are present there) — re-running this script in the current state of the repository would fail on the `Image()` call. This makes the script currently non-portable and, as of this analysis, not runnable without first restoring or re-pointing that logo file.

The remaining files — `README.md` and `.gitignore` at the project root, and `.gitkeep` placeholders inside `storage/`, `uploads/logos/`, and `uploads/avatars/` — are documentation/repository-hygiene files with no runtime behavior. `README.md` is analyzed for accuracy in Part 13; `.gitignore` correctly excludes runtime-generated content (error logs, uploaded logos/avatars) while preserving the directories themselves via `.gitkeep`, which was verified directly (uploaded files are excluded; the directory placeholders are tracked).

Part 3 — Module Analysis

This part explains what each module contributes to the system as a whole, and traces the real, code-verified relationships between the files inside it — including where the actual implementation differs from the "textbook" asset-lifecycle flow.

3.1 Assets Module

modules/assets/list.php, add.php, edit.php, view.php, and _form.php together form the master registry. add.php and edit.php both include the same _form.php partial (the one deliberate view-partial reuse in the codebase), so the create and edit forms cannot drift out of sync with each other. view.php is the hub: it independently queries all five history tables (asset_assigned, asset_transfers, asset_maintenance, asset_audits, asset_disposals) filtered to one asset_id, which is how every other module's history becomes visible without those modules needing to know about each other.

The Real Asset Lifecycle (as implemented, not assumed)

```
Create Asset (assets/add.php)
| INSERT INTO assets
v
Asset Record exists, status = 'active' by default
|
+--> Assign Asset (assigned/add.php) -- asset_assigned row + assets.department_id updated
| |
| +--> Mark Returned / Send to Repair (assigned/return.php) -- closes the allocation row
| (does NOT change assets.status by itself)
|
+--> Transfer Asset (transfers/add.php) -- asset_transfers row + assets.department_id updated
|
+--> Report Maintenance Issue (maintenance/add.php)
| | INSERT asset_maintenance (status='pending')
| v
| recomputeAssetStatus() --> assets.status = 'under_repair'
| |
| v
| Update Maintenance (maintenance/update.php) status -> 'completed'
| |
| v
| recomputeAssetStatus() --> assets.status back to 'active' (if no other open tickets
| and no approved disposal)
|
+--> Record Audit (audits/add.php) -- asset_audits row only; never touches assets.status
|
+--> Request Disposal (disposals/add.php) -- asset_disposals row, status='pending'
|
v
Approve/Reject Disposal (disposals/approve.php) -- Admin / Top Management only
| on 'approved':
v
recomputeAssetStatus() --> assets.status = 'disposed' (this always wins over
'under_repair' -- see includes/functions.php)
```

Two points where the real flow differs from the "textbook" Create → Assign → Transfer → Maintain → Audit → Dispose sequence implied by a typical sidebar ordering: Audits never change assets.status at all (confirmed by reading modules/audits/add.php — a "damaged"/"missing" result is purely informational unless a human separately opens a maintenance ticket or disposal request based on it), and assets.status is not a simple state machine walked in order — it is recomputed from scratch every time

(recomputeAssetStatus() in includes/functions.php), checking "is there an approved disposal?" first, then "is there an open maintenance ticket?", defaulting to active otherwise. An asset can be assigned, transferred, and audited any number of times, in any order, without ever affecting its status.

3.2 Authentication Module

modules/auth/login.php and logout.php, backed entirely by includes/auth.php for the session/role guard functions used everywhere else. There is no registration page and no self-service password reset anywhere in the codebase — every account is created by an Admin via modules/users/add.php.

3.3 Users Module

modules/users/add.php, edit.php, list.php, and deactivate.php form a complete CRUD+state-toggle set, Admin-only. Deactivating a user (deactivate.php) does not delete anything — it flips users.status to 'inactive', which modules/auth/login.php then checks and refuses even with a correct password. This preserves every historical record (activity_logs, asset_assigned.assigned_by, etc.) that references the user's user_id, which a hard delete would either break or cascade-destroy.

3.4 Departments, Categories, Locations Modules

modules/departments/list.php, modules/categories/list.php, and modules/locations/list.php are three structurally identical Admin-only CRUD modules (same create/update/delete-via-modal pattern, same try/catch around DELETE to turn a foreign-key violation into a friendly flash message instead of a raw SQL error). departments/list.php is the most consequential of the three: department_id, set here, is the value every "\$isHead" department-scoping check across the entire application compares against.

3.5 Requisitions Module

```
Submit Requisition (requisitions/add.php) -- Admin or Department Head
| INSERT requisitions (status='pending')
v
Review (requisitions/review.php) -- Admin or Asset Officer
|
+--> decision='approved' -----+--> decision='rejected'
| | |
| v v
| status='approved' status='rejected' (dead end --
| | 'No further action available')
| v
| Mark Issued (requisitions/review.php, same file, decision='issued')
| |
| v
| status='issued' (dead end)

Enforced by $allowedTransitions in review.php: pending -> {approved, rejected} only;
approved -> {issued} only. Any other transition is rejected server-side.
```

A requisition never creates an assets row automatically — "Issued" only records that the department's need has been fulfilled; if new equipment was purchased to satisfy it, an Asset Officer/Admin registers it separately via modules/assets/add.php. Confirmed: no INSERT INTO assets exists anywhere in the requisitions module.

3.6 Disposals Module

```
Request Disposal (disposals/add.php) -- Admin or Asset Officer
| INSERT asset_disposals (status='pending')
v
Review (disposals/approve.php) -- Admin or Top Management ONLY (not the requester)
|
+--> decision='approved' +--> decision='rejected'
| | |
| v v
| status='approved', disposal_date=today status='rejected'
| |
| v
| recomputeAssetStatus() --> assets.status = 'disposed'
```

Separation of duties, confirmed in code: the Asset Officer role that can submit a disposal request is explicitly excluded from `requireRole([ROLE_ADMIN, ROLE_TOPMGMT])` in `approve.php`.

3.7 Maintenance Module

```
Report Issue (maintenance/add.php) -- Admin, Officer, or Department Head (own dept only)
| INSERT asset_maintenance (status='pending')
v
recomputeAssetStatus() --> assets.status usually becomes 'under_repair'
v
Update (maintenance/update.php) -- Admin or Officer only
| status: pending -> in_progress -> completed
| (completed requires a Completed Date; Cost/Technician optional at any stage)
v
recomputeAssetStatus() runs again after every save
--> once every open ticket for the asset is completed, status returns to 'active'
(unless an approved disposal exists, which always takes priority)
```

3.8 Transfers Module

`modules/transfers/add.php` and `list.php`. A transfer is a lighter-weight action than an allocation: it simply moves `assets.department_id` and logs the move, with no separate "return" concept. Guarded against two specific mistakes at the database-query level: transferring an already-disposed asset, and "transferring" an asset to the department it is already in.

3.9 Reports Module

`modules/reports/index.php` (on-screen), `export_csv.php`, and `export_pdf.php`. `index.php` contains its own query logic for the on-screen view (including the per-row asset-detail queries that back the clickable Asset Count modals); `export_csv.php` and `export_pdf.php` both instead call the single shared `includes/functions.php: getReportRows()` — meaning the two export formats are guaranteed to return identical data for identical filters, since they are, literally, the same function call.

3.10 Settings Module

Six admin-only pages sharing modules/settings/_subnav.php: General (index.php), Email/SMTP (smtp.php), Backup & Restore (backup.php + backup_download.php + backup_restore.php), Activity Logs (logs.php), Login Logs (login_logs.php), and System Info (system_info.php). These are independent of each other except that every other module in the system reads values that General writes, via getSetting().

3.11 Dashboard Module

A single file, modules/dashboard/index.php, that queries six independent counts/breakdowns and a 12-row recent-activity feed, all scoped the same way every list page is scoped (\$isHead / department_id). It writes nothing to the database — a pure read/aggregate view.

3.12 Profile Module

A single file, modules/profile/index.php, that is the only place in the system where a non-Admin user can change anything about their own account (name/email/username/profile picture/password). It reuses the exact same email/username uniqueness-check pattern as modules/users/edit.php, applied to the currently-logged-in user's own row instead of an arbitrary one.

Part 4 — Database Relationships

All 15 tables, their primary keys, and every foreign key relationship, read directly from database/schema.sql.

Table	Primary Key	Foreign Keys (-> Referenced Table, ON DELETE rule)
roles	role_id	(none — referenced by users.role_id)
departments	department_id	head_id -> users (SET NULL) — added via ALTER TABLE after users exists
users	user_id	role_id -> roles (RESTRICT); department_id -> departments (SET NULL)
categories	category_id	(none)
locations	location_id	(none)
assets	asset_id	category_id -> categories (RESTRICT); location_id -> locations (SET NULL); department_id -> departments (SET NULL)
asset_assigned	assign_id	asset_id -> assets (CASCADE); assigned_to -> departments (RESTRICT); assigned_user_id -> users (SET NULL); assigned_by -> users (RESTRICT)
asset_transfers	transfer_id	asset_id -> assets (CASCADE); from_department_id -> departments (SET NULL); to_department_id -> departments (RESTRICT); handled_by -> users (RESTRICT)
asset_maintenance	maintenance_id	asset_id -> assets (CASCADE); reported_by -> users (RESTRICT)
asset_audits	audit_id	asset_id -> assets (CASCADE); audited_by -> users (RESTRICT)
requisitions	requisition_id	department_id -> departments (CASCADE); requester_id -> users (RESTRICT); category_id -> categories (SET NULL); reviewed_by -> users (SET NULL)
asset_disposals	disposal_id	asset_id -> assets (CASCADE); requested_by -> users (RESTRICT); approved_by -> users (SET NULL)
settings	setting_key (string)	(none)
activity_logs	log_id	user_id -> users (SET NULL)
login_logs	login_log_id	user_id -> users (SET NULL)

Status fields, confirmed by ENUM definition: users.status (active/inactive); assets.status (active/under_repair/disposed — derived, see Part 3.1); asset_assigned.status (active/returned/repair); asset_maintenance.status (pending/in_progress/completed); asset_audits.result (found/missing/damaged); requisitions.status (pending/approved/rejected/issued); asset_disposals.status (pending/approved/rejected); login_logs.status (success/failed).

Relationship Map (as implemented)

```
roles -----< users >----- departments >---- head_id (self-referential
| | back to users)
| |
| (author/actor FK on | (department_id scoping FK on nearly
| almost every table below) | every table below)
v v
+-----+
| |
v v
categories >---+ locations >---+
| |
v v
assets <-----+
| ^ ^ ^ ^ ^
| | | | |
+-----+ | | | +-----+
| | | | |
v | | | v v
asset_assigned | | | asset_audits asset_disposals >--- requested_by / approved_by -> users
(assigned_to, | | | (audited_by -> users) ^
assigned_user_id, | | |
assigned_by | | | asset_disposals.status='approved'
-> users/depts) | | | is what recomputeAssetStatus() checks
| | | FIRST when deriving assets.status
asset_transfers |
(from/to_department_id -> departments, asset_maintenance
handled_by -> users) (reported_by -> users;
open tickets are what
recomputeAssetStatus() checks
SECOND)

requisitions (department_id -> departments, requester_id / reviewed_by -> users,
category_id -> categories) -- NOT linked to assets at all; a requisition
is a request, never a direct row-level pointer to an asset record.

settings, activity_logs, login_logs -- system/audit tables; activity_logs and login_logs
both point back to users.user_id (SET NULL if the user is later removed).
```

One relationship worth calling out explicitly because it is easy to assume otherwise: **requisitions has no foreign key to assets at all**. It only ever points to departments, users, and categories. The connection between "a requisition was issued" and "an asset now exists" is a manual, human step (a separate modules/assets/add.php submission), not an enforced database relationship — confirmed by inspecting both the requisitions table definition and every requisitions/*.php file.

Part 5 — Role & Permission Analysis

Exactly four roles exist, defined as constants in `includes/auth.php`: **Admin**, **Asset Officer**, **Department Head**, and **Top Management**. Every restriction below was read directly from the ``requireRole([...])`` / ``hasRole([...])`` / ``$isHead`` checks in each module's own PHP file — not inferred from the sidebar.

5.1 Per-Role Summary

Role	Can See	Can Create	Can Edit	Can Approve	Can Delete/Deactivate	Reports	Settings
Admin	Everything, system-wide	Assets, allocations, transfers, maintenance, audits, requisitions (any dept.), disposals, users, master data	Everything	Requisitions, disposals	Users (deactivate); categories/departments/locations (delete)	Full, all data	Full
Asset Officer	Everything, system-wide	Assets, allocations, transfers, maintenance, audits, disposal requests	Assets, maintenance tickets	Requisitions (not disposals — cannot approve their own request)	—	Full, all data	—
Department Head	Own department only (assets, allocations, transfers as source/destination, maintenance, audits, requisitions, disposals)	Maintenance issue reports; requisitions (own department)	— (view-only outside of reporting issues/submitted requisitions)	—	—	Own department scope	—
Top Management	Everything, system-wide, read-only (except disposals)	—	—	Disposals only	—	Full, all data, view-only	—

5.2 Permission Matrix

Feature	Admin	Asset Officer	Department Head	Top Management
Assets	Full	Full	View (own dept.)	View (all)
Users	Full	—	—	—
Departments / Categories / Locations	Full	—	—	—
Allocations	Full	Full	View (own dept.)	View (all)
Transfers	Full	Full	View (source/dest. = own dept.)	View (all)
Maintenance	Full (incl. update/close)	Full (incl. update/close)	Report only + view (own dept.)	View (all)
Audits	Full	Full	View (own dept.)	View (all)
Requisitions	Submit (any dept.) + Review/Approve/Issue	Review/Approve/Issue (cannot submit)	Submit (own dept.) only	View (all)
Disposals	Request + Approve/Reject	Request only	View (own dept.)	Approve/Reject only
Reports	Full, all data + CSV/PDF export	Full, all data + CSV/PDF export	Own department scope + CSV/PDF export	Full, all data, view-only + CSV/PDF export
Settings (all six tabs)	Full	—	—	—
Own Profile / Password	Full	Full	Full	Full

Enforcement is server-side, confirmed for every module read. Every restricted page calls `requireRole(...)` as one of its first lines (before any output), and the sidebar's own role check (`includes/layout/sidebar.php: `if ($role === ROLE_ADMIN)``) matches the equivalent server-side guard exactly everywhere it was checked. A user without the right role who opens a restricted URL directly is shown `includes/layout/forbidden.php` with an HTTP 403 status, not silently redirected — this was verified in `modules/users/*.php`, `modules/settings/*.php`, and `modules/disposals/approve.php`.

Part 6 — Complete System Workflows

6.1 Authentication

```
Login form submitted (modules/auth/login.php, POST)
-> verifyCsrf() fail -> 'Your session expired. Please try again.'
-> loginValue/password both present? no -> 'Username/email and password are both required.'
-> SELECT ... WHERE email = :login1 OR username = :login2
-> password_verify($password, $user['password']) fail -> 'Invalid username/email or password.'
+ logLogin(..., 'failed')
-> $user['status'] === 'active'? no -> 'Your account has been deactivated...'
+ logLogin(..., 'failed')
-> session_regenerate_id(true)
-> populate $_SESSION (user_id, name, email, username, profile_picture, role_id, role_name,
department_id, department_name)
-> logLogin(..., 'success') + logActivity('Login')
-> redirect to $_SESSION['redirect_after_login'] or modules/dashboard/index.php
```

6.2 Asset Management

See the full lifecycle diagram in Part 3.1 — reproduced there because it is the module every other workflow ultimately reports back to.

6.3 Requisition

See Part 3.5 for the exact, code-verified status transition diagram (pending → approved/rejected; approved → issued; both server-enforced).

6.4 Disposal

See Part 3.6. The one workflow in the system with an explicit, code-enforced separation of duties: the role that can request a disposal (Admin/Asset Officer) never overlaps with the role that approves it when the requester is an Asset Officer (Admin/Top Management only).

6.5 Maintenance

See Part 3.7. The one workflow that automatically changes assets.status in both directions (active → under_repair on report, under_repair → active on completion), via recomputeAssetStatus().

6.6 Transfer

See Part 3.8 — a single-step action (no review/approval stage), gated to Admin/Asset Officer, with two validation guards (not a disposed asset; not already in the destination department).

6.7 Reports — Generation and Export

```
modules/reports/index.php (on-screen)
-> $report = by_department | by_category | by_status | maintenance_cost | disposals
-> aggregate GROUP BY query for the table + matching per-row detail query (same scoping)
-> renderAssetCountCell()/renderAssetDetailModal() build the clickable Asset Count UI

CSV Export (modules/reports/export_csv.php) PDF Export (modules/reports/export_pdf.php)
-> includes/functions.php: getReportRows() -> includes/functions.php: getReportRows()
(the SAME function index.php's own queries (the SAME function again)
independently re-derive equivalent data from) -> vendor/fpdf/fpdf.php: ReportPDF (custom
-> fputcsv() to php://output Header/Footer, auto Portrait/Landscape,
-> Content-Disposition: attachment; .csv fitText() truncation)
-> $pdf->Output('D', ...); .pdf download
```

6.8 Backup

```
Settings -> Backup & Restore (modules/settings/backup.php)
-> shows DB size + table count (information_schema query)

Download Backup (modules/settings/backup_download.php)
-> SHOW TABLES -> for each table: SHOW CREATE TABLE + SELECT * -> hand-built INSERT statements
-> streamed directly as a .sql file download (no temp file written to disk)

Restore Database (modules/settings/backup_restore.php)
-> validates .sql extension + <= 50MB
-> $pdo->exec($sql) <-- the ENTIRE uploaded file content, executed as-is against the live DB
-> success/failure flashed back to modules/settings/backup.php

No automatic pre-restore backup is taken. No staging/dry-run step exists. See Part 8, Finding #1.
```

Part 7 — Frontend

UI Structure

Every authenticated page shares one shell, assembled by `includes/layout/header.php` + `sidebar.php` + `footer.php`: a fixed-width, always-dark-navy left sidebar; a sticky white topbar; and a content area in between. There is no client-side routing or single-page-app behavior anywhere — every navigation is a normal full-page HTTP request.

CSS Loading

Exactly one stylesheet, `static/css/style.css`, linked from `includes/layout/header.php` and separately from `modules/auth/login.php` (which does not go through `header.php`, since it renders before a user is authenticated). Both references append `?v=<filetime(>` so a browser never serves a stale cached copy after a CSS edit. The Google Font "Inter" is loaded from Google Fonts via a preconnect + stylesheet link, with a system-font fallback stack defined in the CSS itself.`

JavaScript Loading

`static/js/main.js` is loaded once, from `includes/layout/footer.php` (also cache-busted), and covers every page. `static/js/validation.js` is loaded separately, only by `modules/auth/login.php`. No other module loads any additional script — all interactivity elsewhere is handled by `main.js`'s generic, attribute-driven handlers.

Navigation / Sidebar

`includes/layout/sidebar.php` renders a fixed list of links via a local `navItem()` helper, wrapping each in an `<a>` with an "active" class match against `$activeMenu` (set by the including module). The "Administration" section (Users/Departments/Categories/Locations/Settings) is wrapped in a single ``if ($role === ROLE_ADMIN)`` check — the only role-conditional rendering block in the sidebar.

Forms

Every form in the system follows the same pattern: a `<?= csrfField() ?>` hidden input, ``method="post"`, and server-side validation via includes/functions.php: validateRequired() plus each module's own extra checks (numeric, email format, enum membership, uniqueness). No form was found that skips the CSRF field.`

Validation

Two layers, confirmed independently: `static/js/validation.js` runs first in the browser (required/email/number-range/maxlength checks, purely for immediate feedback), and every POST handler re-validates from scratch server-side regardless of what the client already checked — the client-side layer could be disabled entirely and no invalid data could reach the database as a result.

Responsive Behavior

Two CSS breakpoints in `static/css/style.css`: at 992px the sidebar becomes an off-canvas panel toggled by a hamburger button (`#sidebarToggle`, wired in `main.js`), and at 600px content padding and heading sizes shrink further. No separate mobile stylesheet or JS exists.

Which Modules Use Which Frontend Resources

Resource	Used By
static/css/style.css	Every page in the system, including the pre-auth login and 403 pages
static/js/main.js	Every authenticated page (via footer.php)
static/js/validation.js	modules/auth/login.php only
static/images/logo.webp	includes/layout/sidebar.php + header.php + modules/auth/login.php, as the fallback whenever no admin-uploaded logo is set
Google Fonts "Inter"	Every authenticated page + the login page (both load it independently)

Part 8 — Authentication & Security

Every finding below was produced by reading the actual implementation, not by assumption. Nothing in this section has been changed — this is analysis only, as requested.

8.1 What Is Implemented Correctly (confirmed by reading the code)

Control	Verified Implementation
Password hashing	password_hash()/password_verify() (bcrypt) — no plain-text or reversible password storage found anywhere.
SQL injection	PDO prepared statements with bound parameters used in every query across all ~50 module files inspected; PDO::ATTR_EMULATE_PREPARES is explicitly false (real server-side prepares). The one exception is modules/settings/backup_restore.php, which is a deliberate raw-SQL feature, not an injection bug — see Finding #1.
XSS	includes/functions.php: e() (htmlspecialchars, ENT_QUOTES) wraps essentially all user-influenced output in every module's HTML; no raw-echoed user input was found.
CSRF	includes/functions.php: csrfField()/verifyCsrf()/requireCsrf(), a random 32-byte token compared with hash_equals(); present on every state-changing form found in the codebase.
Session hardening	config/config.php: session.use_strict_mode, HttpOnly, SameSite=Lax, and Secure-when-HTTPS cookie flags; session_regenerate_id(true) on every successful login (session fixation protection).
Role enforcement	Server-side requireRole()/hasRole() on every restricted page — confirmed consistent with the sidebar's own role checks (see Part 5).
Audit trail	logActivity()/logLogin() called from every state-changing action reviewed, feeding the Settings → Activity/Login Logs pages.

8.2 Findings

Finding #1: Unrestricted SQL execution via database restore

CRITICAL

Location

modules/settings/backup_restore.php

Problem

`\$pdo->exec(\$sql)` executes the entire contents of an uploaded file directly against the live database. The only checks are the file extension (.sql) and a 50MB size cap — the SQL itself is not parsed, filtered, or limited to safe statement types in any way.

Risk

Any Admin session — the legitimate administrator, a mistakenly-uploaded wrong file, or an attacker who has obtained an Admin session (e.g. via a stolen session cookie, or a phished credential, since no second factor exists) — can drop, recreate, or arbitrarily repopulate every table in one request, with no automatic safety net taken beforehand.

Recommended Fix (not applied — analysis only)

Take an automatic on-disk backup immediately before every restore; parse and constrain the uploaded file to expected DDL/DML patterns (or require it to have been produced by this system's own backup_download.php, e.g. via a signed header); consider requiring a second confirmation step showing a diff/summary of what will change before executing.

Finding #2: No login rate limiting or account lockout

MEDIUM

Location

modules/auth/login.php

Problem

Every failed attempt is logged (login_logs), but nothing reads that history to slow down or block repeated attempts against the same account or from the same IP address.

Risk

A password can be brute-forced or credential-stuffed at whatever rate the network allows, with the only trace being a login_logs table that no part of the running application ever inspects automatically.

Recommended Fix (not applied — analysis only)

Track recent failed attempts per account/IP (the login_logs table already has everything needed) and add a short lockout or increasing delay after a threshold; optionally surface repeated-failure alerts to the Admin.

Finding #3: Session timeout setting is not enforced

MEDIUM

Location

config/config.php + modules/settings/index.php

Problem

The session cookie lifetime is set once, in config/config.php, from a hardcoded fallback (1440 minutes) before the database — and therefore the settings table — is even reachable. The "Session Timeout (minutes)" field in Settings → General is validated and saved to the settings table, but nothing in the codebase re-applies it to session_set_cookie_params() or an idle-timeout check on subsequent requests.

Risk

Not a vulnerability by itself (the fallback is a fixed, reasonable 24 hours), but it is a silently-non-functional admin control: changing this setting has zero observable effect, which could give an administrator false confidence that sessions expire sooner than they actually do.

Recommended Fix (not applied — analysis only)

Either enforce it (e.g. store the configured minutes in the session and check elapsed idle time on each request in bootstrap.php) or remove the setting from the UI until it is wired up.

Finding #4: Uploaded files are validated by extension only, not content

MEDIUM

Location

modules/profile/index.php (avatars) and modules/settings/index.php (logos)

Problem

Both uploaders check the file extension against a whitelist (jpg/jpeg/png/gif) and a size cap, but neither verifies the file is actually an image (e.g. via getimagesize() or a MIME-type/magic-byte check). No .htaccess or web-server rule was found anywhere in the repository restricting script execution inside uploads/avatars/ or uploads/logos/.

Risk

If the web server were ever configured to execute PHP from the uploads directory (not the case in a default XAMPP docroot, but a realistic misconfiguration on other hosts), a file with a spoofed .jpg/.png extension but PHP content could be uploaded and, if directly requested, executed.

Recommended Fix (not applied — analysis only)

Validate actual image content (getimagesize() or finfo_file() MIME detection) in addition to the extension check, and add an uploads/.htaccess (or equivalent web-server config) explicitly denying script execution in that directory as defense-in-depth.

Finding #5: Database credentials committed in plain text

LOW (in the current local/demo context) / HIGH if unchanged before a real deployment

Location

config/database.php

Problem

DB_HOST/DB_NAME/DB_USER/DB_PASS are literal constants in a version-controlled file (root user, empty password in this project's local/demo configuration).

Risk

Low risk for the local/demo environment this project ships with, but if the same file or pattern were carried into a real deployment with a non-empty, meaningful password, that password would be exposed to anyone with repository access.

Recommended Fix (not applied — analysis only)

Externalize credentials (environment variables or a git-ignored local config file) before any non-local deployment; keep config/database.php as a checked-in template with placeholder values only.

Finding #6: Duplicate named PDO placeholder crashes the Users search box

MEDIUM

Location

modules/users/list.php

Problem

The search query — `(u.name LIKE :q OR u.email LIKE :q OR u.username LIKE :q)` — binds the same named placeholder `:q` three times in one prepared statement. Because config/database.php sets PDO::ATTR_EMULATE_PREPARES to false, MySQL's native prepared-statement protocol rejects a repeated named placeholder, throwing `SQLSTATE[HY093]: Invalid parameter number`.

Risk

Any Admin who types a search term into Users → Search currently receives an HTTP 500 error instead of filtered results — a functional availability bug, not a data-exposure risk, but it is a currently-reproducible defect in an Admin-only screen. (The identical bug class was previously found and fixed in modules/auth/login.php and modules/assets/list.php, using distinct placeholder names for each repeated occurrence — that fix pattern was not carried over to this file.)

Recommended Fix (not applied — analysis only)

Bind three distinct placeholders (e.g. :q1/:q2/:q3) to the same search value, exactly as was already done for the equivalent fix in modules/assets/list.php.

Finding #7: Backup downloads contain every user's password hash and the raw SMTP password

LOW

Location

modules/settings/backup_download.php

Problem

The generated .sql dump includes full INSERT statements for the users table (bcrypt hashes) and the settings table (which can contain a plain-text smtp_password value if one has been configured).

Risk

This is expected/necessary for a functional database backup, but it means the downloaded file itself is exactly as sensitive as the live database and must be stored/transmitted with equivalent care — a risk of the feature's nature rather than a coding mistake.

Recommended Fix (not applied — analysis only)

Document this clearly for administrators; consider encrypting the download or requiring a fresh re-authentication step immediately before it is generated.

8.3 Not Applicable / Not Found In This Codebase

The system has no file-execution-from-upload path outside of the two upload directories already covered in Finding #4 (no arbitrary file-write endpoint elsewhere was found). No client-side secrets (API keys, tokens) are embedded in static/js/*.js. No third-party JavaScript beyond the Google Fonts stylesheet link is loaded. No cookies other than the PHP session cookie are set anywhere in the codebase.

Part 9 — Reports & PDF

Covers `modules/reports/*`, `vendor/fpdf/*`, and `docs/build_pdf.py`, read in full.

How Reports Are Generated (on screen)

`modules/reports/index.php` runs one of five report branches based on a ``report`` query-string value, each pairing a GROUP BY aggregate query with a per-row detail query using identical WHERE scoping — this is what guarantees the clickable Asset Count modal always shows the exact records the count was computed from (verified programmatically during development: the modal row count and the on-screen number match exactly for every department/category/status, and the sum of each modal's per-asset Value column reconciles to the row's displayed Total Value).

How CSV Export Works

`modules/reports/export_csv.php` calls `includes/functions.php: getReportRows($pdo, $report, $isHead, $deptId, $dateFrom, $dateTo)` — the same shared function PDF export also calls — then streams the result through PHP's built-in `fputcsv()` directly to `php://output` with a Content-Disposition: attachment header. No temporary file is written to disk.

How PDF Export Works

`modules/reports/export_pdf.php` also calls `getReportRows()`, then builds the document with a small `ReportPDF` class extending `vendor/fpdf/fpdf.php`'s `FPDF`. It overrides `Header()` (university name + report title + generation timestamp, centered) and `Footer()` (page N/total, using `FPDF`'s `AliasNbPages()`), and adds one local helper, `fitText()`, that measures each cell's rendered width with `GetStringWidth()` and truncates with an ellipsis rather than letting any cell overflow into its neighbor. Orientation is chosen automatically: Landscape once a report has four or more columns (Maintenance Cost, Disposals), Portrait otherwise.

Which Data Is Used

Exactly the same five report definitions in both export formats and the on-screen view — by construction, since `export_csv.php` and `export_pdf.php` both delegate to `getReportRows()` rather than maintaining any query logic of their own.

Which Library Is Used, and How It Is Connected

`FPDF` (vendored by hand at `vendor/fpdf/fpdf.php` — no Composer, consistent with the rest of the project). It is required directly with ``require_once APP_ROOT . '/vendor/fpdf/fpdf.php`` at the top of `export_pdf.php`; there is no autoloader, and no other file in the project references `FPDF` at all.

`docs/build_pdf.py` — A Separate, Offline Tool

Not part of the running web application and not connected to `modules/reports/` in any way. It is a standalone Python/reportlab script, run manually by a developer, that produces the existing end-user/admin manual PDF already present in `docs/`. See Part 2.21 for the specific portability issue found in this script (a hardcoded, currently-missing logo path).

Part 10 — Settings

modules/settings/ contains six admin-only pages sharing modules/settings/_subnav.php for consistent tab navigation.

Tab (file)	Purpose	Relationship to the Rest of the System
General (index.php)	University/system name, academic year, language, timezone, date format, session timeout, records-per-page, theme, logo upload, maintenance-mode switch.	The single source for every includes/functions.php: getSetting() call made anywhere else in the app — university name in every page title/footer, theme applied to every <body> class, language applied globally via includes/i18n.php, records-per-page used by every paginated list.
Email / SMTP (smtp.php)	SMTP host/port/username/password/encryption for outgoing email.	Currently a dead end — no code anywhere in the project reads these settings to actually send an email (see Part 13).
Backup & Restore (backup.php, backup_download.php, backup_restore.php)	Full database dump download and raw-SQL restore-from-upload.	Self-contained; does not feed any other module. The system's only backup mechanism — see Part 8, Finding #1 for the associated risk.
Activity Logs (logs.php)	Paginated view of activity_logs, filterable by module.	Every logActivity() call made by any module (create/update/delete/state-change actions) lands here.
Login Logs (login_logs.php)	Paginated view of login_logs, filterable by success/failed.	Every logLogin() call from modules/auth/login.php lands here — currently observational only, not acted on automatically (see Part 8, Finding #2).
System Info (system_info.php)	Read-only PHP/MySQL/server/disk snapshot.	Purely diagnostic; writes nothing, reads nothing any other module depends on.

Role access, confirmed identically across all six files: every one of them calls `requireRole([ROLE_ADMIN])` as one of its first lines. No other role can reach any Settings page, including by direct URL — each would receive the 403 Access Denied page.

Part 11 — Dependency Map

The include/require chain and the shared-function fan-out, as they actually exist in the code (not an idealized version).

```
EVERY modules/**/*.php file starts with:
require_once __DIR__ . '/../../includes/bootstrap.php';

includes/bootstrap.php
|-- require_once config/config.php (constants, session hardening + session_start())
|-- require_once config/database.php (creates $pdo, or dies with HTTP 500)
|-- require_once includes/auth.php (ROLE_* constants, requireLogin/requireRole/hasRole)
|-- require_once includes/functions.php (e/clean/csrf*/flash/format*/statusBadge/icon/
| getPendingAlerts/getReportRows/logActivity/logLogin/
| getSetting family/recomputeAssetStatus/validateRequired)
|-- require_once includes/i18n.php (lazily requires includes/i18n_strings.php on first t()
call)
|-- setActiveLanguage(getSetting($pdo, 'language'))
'-- maintenance-mode gate (reads getSetting($pdo, 'maintenance_mode'))

then the module file's own logic runs, typically:
|-- requireLogin() / requireRole([...]) (includes/auth.php)
|-- $pdo->prepare(...)->execute([...]) (config/database.php's $pdo)
|-- on POST: requireCsrf() -> validateRequired()/custom checks -> INSERT/UPDATE/DELETE
| -> logActivity() -> flash() -> redirect()
'-- on GET: builds $pageTitle/$activeMenu, then:
include includes/layout/header.php
|-- getSetting() (university name, theme)
|-- getPendingAlerts() (includes/functions.php)
'-- include includes/layout/sidebar.php
'-- icon()/appLogoUrl()/t() (includes/functions.php, includes/i18n.php)
... the module's own HTML, using e()/statusBadge()/formatMoney()/formatDate() ...
include includes/layout/footer.php
'-- loads static/js/main.js

modules/reports/export_csv.php -----+
+--> includes/functions.php: getReportRows() (single shared query)
modules/reports/export_pdf.php -----+
'--> vendor/fpdf/fpdf.php (PDF rendering only)

modules/assets/add.php --+
modules/assets/edit.php --+--> modules/assets/_form.php (the one shared view partial in the
project)

modules/settings/index.php, smtp.php, backup.php, logs.php, login_logs.php, system_info.php
each --> modules/settings/_subnav.php (shared tab strip)

includes/functions.php: recomputeAssetStatus() <-- called from:
modules/maintenance/add.php, modules/maintenance/update.php, modules/disposals/approve.php
(NOT called from modules/assets/edit.php -- see Part 2.7 / Part 13)
```

Part 12 — What Each File Contributes to the System

A compact, single-row-per-file summary for quick reference. Full detail for every file is in Part 2.

File	What It Does	What It Contributes to ASM
index.php	Redirects to Dashboard or Login based on session state.	The single public entry point.
config/config.php	Constants + session hardening + session_start().	Makes every other file able to run safely.
config/database.php	Opens the shared PDO connection.	Makes every database query possible.
database/schema.sql	15-table schema + seed data.	Defines the entire data model.
includes/bootstrap.php	Wires config/db/auth/functions/i18n together; maintenance-mode gate.	The composition root every page starts from.
includes/auth.php	Role constants + login/role guard functions.	Enforces every access rule in the system.
includes/functions.php	Sanitization, CSRF, formatting, logging, notifications, shared report query.	The most-reused utility layer in the codebase.
includes/i18n.php / i18n_strings.php	Translation lookup + English/Somali dictionary.	Lets the app shell + dashboard + login run in two languages.
includes/layout/header.php	Opens the HTML shell, topbar, notification bell.	Consistent chrome on every page.
includes/layout/sidebar.php	Role-aware navigation menu.	Primary means of moving between modules.
includes/layout/footer.php	Closes the shell, loads main.js.	Consistent chrome + shared interactivity.
includes/layout/forbidden.php	403 Access Denied page.	Makes denied access visible rather than silently hidden.
modules/auth/login.php	Authenticates and starts a session.	The front door of the whole system.
modules/auth/logout.php	Destroys the session.	Clean session termination.
modules/dashboard/index.php	Role-scoped summary stats + activity feed.	At-a-glance system state on login.
modules/assets/list.php	Searchable, paginated asset list.	Primary browsing surface for the asset register.
modules/assets/add.php	Registers a new asset.	Creates the master record everything else attaches to.
modules/assets/edit.php	Edits an asset.	Corrections/reclassification.
modules/assets/view.php	One asset + its full 5-table history.	Single source of truth per asset.
modules/assets/_form.php	Shared add/edit form fields.	Keeps create and edit forms in sync.
modules/assigned/add.php	Issues an asset to a department/custodian.	Starts an allocation.
modules/assigned/list.php	Lists allocations + Return/Repair actions.	Operational "who has what" view.
modules/assigned/return.php	Closes an active allocation.	Ends an allocation cleanly.

File	What It Does	What It Contributes to ASM
modules/transfers/add.php	Moves an asset between departments.	Inter-department movement record.
modules/transfers/list.php	Transfer history.	Read-only audit trail of movement.
modules/maintenance/add.php	Reports a repair issue.	Starts a maintenance ticket; flips status to Under Repair.
modules/maintenance/list.php	Maintenance ticket queue.	Operational repair-tracking view.
modules/maintenance/update.php	Progresses/closes a ticket.	Closes the repair loop; restores Active status.
modules/audits/add.php	Records a physical stock-check result.	Physical verification evidence.
modules/audits/list.php	Audit history.	Read-only verification trail.
modules/disposals/add.php	Requests an asset write-off.	Starts the disposal workflow.
modules/disposals/approve.php	Approves/rejects a disposal.	The only place an asset becomes Disposed.
modules/disposals/list.php	Disposal request history.	Full write-off decision trail.
modules/requisitions/add.php	Submits a department request.	Starts the requisition workflow.
modules/requisitions/list.php	Requisition queue.	Review/tracking surface.
modules/requisitions/review.php	Approves/rejects/issues a requisition.	Enforces the requisition status workflow.
modules/categories/list.php	Category CRUD.	Feeds Assets/Requisitions category dropdowns.
modules/departments/list.php	Department CRUD + Head assignment.	Defines the scope every dept.-locked view uses.
modules/locations/list.php	Location CRUD.	Feeds the Assets location dropdown.
modules/users/add.php	Creates a user account.	The only way new accounts are provisioned.
modules/users/edit.php	Edits a user, role, department, password.	Account/role administration.
modules/users/list.php	Searchable user list + Deactivate.	Account administration hub.
modules/users/deactivate.php	Toggles active/inactive.	Revokes access without deleting history.
modules/profile/index.php	Self-service profile + password.	Lets any user maintain their own account.
modules/reports/index.php	Five on-screen reports + drill-down modals.	The system's analytics surface.
modules/reports/export_csv.php	Streams a report as CSV.	Spreadsheet-compatible export.
modules/reports/export_pdf.php	Streams a report as PDF.	Printable/archivable export.
modules/settings/index.php	General system configuration.	The source of every global setting used app-wide.
modules/settings/smtp.php	Saves SMTP credentials.	Currently unused by any sending code (see Part 13).
modules/settings/backup.php	Backup/restore landing page.	Entry point for the system's only backup mechanism.

File	What It Does	What It Contributes to ASM
modules/settings/backup_download.php	Streams a full .sql dump.	The system's only backup output.
modules/settings/backup_restore.php	Executes an uploaded .sql file.	The system's only restore mechanism (see Part 8).
modules/settings/logs.php	Activity log viewer.	System-wide audit trail UI.
modules/settings/login_logs.php	Login attempt viewer.	Authentication forensics UI.
modules/settings/system_info.php	Environment snapshot.	Diagnostic reference for support/evaluation.
modules/settings/_subnav.php	Shared Settings tab strip.	Visual/structural consistency across Settings.
static/css/style.css	The entire visual theme (light + dark).	Consistent look across every page.
static/js/main.js	Sidebar/dropdowns/modals/confirm/search/sort/tabs.	All shared client-side interactivity.
static/js/validation.js	Client-side form validation.	UX convenience layer on the login form.
static/images/logo.webp	Default university logo.	Brand identity when no custom logo is uploaded.
vendor/fpdf/fpdf.php	Third-party PDF rendering engine.	Powers Reports → Export PDF.
docs/build_pdf.py	Generates the end-user/admin manual PDF.	Project documentation tooling (offline, not part of the app).

Part 13 — Missing or Weak Areas

Documented only — nothing below has been changed.

Confirmed Bugs

- **Users search crashes (HTTP 500).** `modules/users/list.php` reuses the named placeholder ``:q`` three times in one query under a PDO configuration that forbids it. See Part 8, Finding #6.

Incomplete Workflows

- **SMTP settings are saved but never used.** `modules/settings/smtp.php` collects and persists full email-server credentials, and its own help text promises "outgoing notification emails (e.g. maintenance updates, requisition decisions)" — but no code anywhere in the repository (no `mail()`, no `PHPMailer`, no other mailer) ever sends an email. This is a configured-but-nonfunctional feature.
- **No self-service password reset.** Only an Admin can reset another user's password (via `modules/users/edit.php`). There is no "forgot password" flow, which would naturally depend on the still-unused SMTP settings to deliver a reset link/code.
- **Session timeout setting has no effect.** See Part 8, Finding #3.
- **Internationalization covers only the app shell.** `i18n_strings.php`'s own header comment states this: only the sidebar/topbar/footer, login page, dashboard, and Settings → General are translated. All ~40 CRUD module pages (Assets, Allocations, Transfers, Maintenance, Audits, Disposals, Requisitions, Users, Departments, Categories, Locations, the other five Settings tabs, Reports) remain English-only regardless of the selected language.
- **Notification bell has no persisted read/unread state.** `includes/functions.php: getPendingAlerts()` recomputes "your request was decided" items live from a 7-day lookback window on every page load; there is no notifications table, so an item silently stops appearing after 7 days whether or not the user ever saw it, and there is no dedicated "notification history" page.

Data-Consistency Risk

- **Manually-edited asset status can drift.** `modules/assets/edit.php` allows a direct, unrestricted edit of `assets.status`, while the rest of the system treats that field as derived (`recomputeAssetStatus()` in `includes/functions.php`, called only from the maintenance and disposal modules). A hand-set status persists only until the next maintenance/disposal event silently overwrites it — the edit form warns about this in a caption but does not prevent it.

Workflow Design Asymmetry (worth confirming against the intended business process, not necessarily a bug)

- Disposals require Top Management (or Admin) approval — a strict separation of duties. Requisitions, by contrast, can be fully approved/rejected/issued by an Asset Officer alone, with no Department Head or Top Management sign-off, even though both are department-driven requests with financial impact. This may be intentional (requisitions are lower-stakes than write-offs) but is a real, code-confirmed asymmetry in how the two workflows are governed.

Missing Functionality / Coverage Gaps

- No pagination on the Allocations, Transfers, Maintenance, Audits, Disposals, Requisitions, Categories, Departments, or Locations lists — only Assets and the two Settings log viewers paginate. Every other list loads its full result set in one query, which will degrade as those tables grow.
- No automated tests exist anywhere in the repository (no `/tests` directory, no `PHPUnit`/other test framework configuration).

- No API/JSON layer — the system is entirely server-rendered HTML; integrating a mobile app or external system would require building a new interface from scratch.
- No explicit web-server rule (e.g. .htaccess) was found restricting access to storage/php-error.log, which could reveal internal file paths and error details if the web server is configured to serve that directory's contents directly.

Duplicate / Poorly Organized Code

- modules/reports/index.php independently re-implements query logic that is very close to (but not the same call as) includes/functions.php: getReportRows() — the on-screen view and the two export formats compute equivalent data through two separate code paths rather than one, which is a minor duplication risk (a future change to one query shape would need to be mirrored in the other by hand).
- docs/build_pdf.py hardcodes an absolute Windows path to a logo file that does not currently exist in the repository, making the script non-runnable as-is (see Part 2.21).

Not Found / Confirmed Absent (stated explicitly, per the request not to guess)

- No evidence of a scheduled/automatic backup job (e.g. cron) anywhere in the codebase — backups are always a manual, on-demand action via Settings.
- No rate limiting, WAF, or similar infrastructure-level protection is configured within the application code itself — cannot be confirmed from the available code whether such protection exists at the hosting/infrastructure layer outside this repository.
- Cannot be confirmed from the available code whether the production deployment (if any exists beyond this local/demo environment) uses different, non-default database credentials or HTTPS — nothing in this repository specifies a production hosting configuration.

Part 14 — Final Architecture Diagram

```
UNIVERSITY ASSET MANAGEMENT SYSTEM (ASM)
|
+-----+
| | |
FRONTEND APPLICATION LOGIC DATA LAYER
(static/, layout) (includes/, modules/) (database/schema.sql)
| | |
static/css/style.css includes/bootstrap.php <--- composition root
static/js/main.js | |
static/js/validation.js +-- config/config.php |
static/images/logo.webp +-- config/database.php -----+--> PDO -> MySQL/MariaDB
includes/layout/*.php +-- includes/auth.php | (15 InnoDB tables,
(shared HTML shell, +-- includes/functions.php | utf8mb4, FK-enforced)
role-aware sidebar, +-- includes/i18n.php / strings |
notification bell) | |
v |
modules//.php |
(one file = controller + view, |
14 feature modules: auth, dashboard, |
assets, assigned, transfers, maintenance, |
audits, disposals, requisitions, users, |
departments, categories, locations, |
reports, settings, profile) -----> all reads/writes go
| through PDO prepared
v statements only
Rendered HTML response
(same file, no template engine)

THIRD-PARTY DEPENDENCY
vendor/fpdf/fpdf.php <-- used only by modules/reports/export_pdf.php

RUNTIME / GENERATED CONTENT
uploads/logos, uploads/avatars, storage/ (error log + reserved backups/)

Request flow: Browser -> module file -> bootstrap.php
(config+db+auth+functions+i18n+maintenance gate)
-> role guard -> business logic (PDO) -> shared header/sidebar/footer -> HTML response.
```

Final Recommendations

Priority Order

- **1 (Critical).** Add a pre-restore automatic backup and tighten what modules/settings/backup_restore.php will execute (Finding #1).
- **2 (High).** Fix the Users search crash by using distinct bound placeholders, the same pattern already applied to the login and Assets fixes (Finding #6).
- **3 (Medium).** Decide, and then either build or remove: email notifications (the SMTP settings already exist and are simply unconnected) and self-service password reset that would naturally build on it.
- **4 (Medium).** Either enforce the Session Timeout setting or remove it from the Settings UI so it stops implying a control that does not exist (Finding #3).
- **5 (Medium).** Add real image-content validation and an uploads/ execution-lockdown rule as defense-in-depth around the two file-upload features (Finding #4).
- **6 (Low/ongoing).** Extend i18n_strings.php coverage to the remaining CRUD modules if full Somali coverage is a goal; add pagination to the currently-unpaginated list pages before they grow large enough to matter; consider whether the Requisitions approval workflow should require the same higher-level sign-off Disposals already has.

Complete Summary

The University Asset Management System is a deliberately framework-free PHP/PDO/MySQL application with a consistent, easy-to-follow structure: one bootstrap file every page loads first, one shared helper library, one shared HTML shell, one shared CSS file, one shared JS file, and one module folder per feature area, each file acting as its own controller and view. Role-based access control is enforced server-side and consistently, in every module checked. The core asset lifecycle — registration, allocation, transfer, maintenance, audit, requisition, and disposal — is fully implemented and its status-derivation logic (`recomputeAssetStatus()`) is centralized in one function. Security fundamentals (prepared statements, output escaping, CSRF, password hashing, session hardening) are applied consistently across the codebase that was read for this analysis, with one clearly-scoped critical risk (the unrestricted SQL restore feature) and a handful of medium/low findings, all documented above with concrete locations and fixes. The system's weakest areas are less about incorrect code and more about incomplete features that are visibly promised but not wired up — SMTP-backed notifications chief among them — plus one currently-reproducible bug in the Users search box that mirrors an already-fixed pattern elsewhere in the same codebase.